

Go (Golang)

SDE2 Interview Guide

100 Questions & Answers with Code Examples

Level: Mid (2–4 yrs)	Stack: Go / Golang	Roles: SDE2 / Backend
----------------------	--------------------	-----------------------

#	Topic	Questions
1	Goroutines & Concurrency	Q1–Q15
2	Channels & Select	Q16–Q28
3	Runtime & Memory	Q29–Q40
4	Interfaces & Types	Q41–Q52
5	Error Handling	Q53–Q60
6	Context	Q61–Q67
7	Generics	Q68–Q74
8	Testing & Benchmarking	Q75–Q82
9	Go Patterns	Q83–Q92
10	Miscellaneous & Advanced	Q93–Q100

100 Questions & Answers with Diagrams and Trade-offs

> **Prepared for:** SDE2 role interviews | **Focus:** Distributed Systems, Scalability, Architecture | **Level:** Mid-level (2–4 years)

Table of Contents

[Databases & Storage](#2-databases--storage) — Q13–Q25
[Caching & CDN](#3-caching--cdn) — Q26–Q34
[Messaging & Queues](#4-messaging--queues) — Q35–Q43
[Distributed Systems Concepts](#5-distributed-systems-concepts) — Q44–Q56
[Microservices & APIs](#6-microservices--apis) — Q57–Q65
[Observability & Reliability](#7-observability--reliability) — Q66–Q73
[Classic System Designs](#8-classic-system-designs) — Q74–Q87
[Advanced System Designs](#9-advanced-system-designs) — Q88–Q95
[Real-World Trade-offs](#10-real-world-trade-offs) — Q96–Q100

Fundamentals & Estimation

Q1. How do you approach a system design interview?

Pattern:
Interview
framework

Difficulty: Easy

Use the RESHADED framework: Requirements → Estimation → Schema → High-level design → APIs → Deep dive → Evaluate.

Step 1 – Requirements (5 min)

Functional: What does it do? (core features only)
Non-functional: Scale, latency, availability, consistency
Out of scope: What won't you design?

Step 2 – Estimation (3 min)

DAU, QPS, storage, bandwidth

Step 3 – Data Model / Schema (3 min)

Key entities, relationships, access patterns

Step 4 – High-Level Design (10 min)

Draw boxes: Client → LB → API servers → Cache → DB
Identify bottlenecks

Step 5 – API Design (3 min)

Core endpoints (3-4 max)

Step 6 – Deep Dive (15 min)

Pick 2 hardest components – drill down

Step 7 – Evaluate / Trade-offs (2 min)

What breaks at 10x scale? What would you change?

Interview tip: "Always clarify before designing. Spending 5 minutes on requirements prevents 20 minutes of wasted design. Ask: 'Should I focus on read scale or write scale?'"

Q2. How do you estimate requests per second (QPS)?

Pattern: Back-of-envelope

Difficulty: Easy

Twitter-like system:

DAU = 100 million users

Each user: 5 tweets/day read, 0.1 tweets/day write

Read QPS = $100M \times 5 / 86400 \approx 5,800$ reads/sec $\rightarrow \sim 6K$ reads/sec

Write QPS = $100M \times 0.1 / 86400 \approx 115$ writes/sec $\rightarrow \sim 120$ writes/sec

Peak QPS = avg $\times 3 \approx 18K$ reads/sec

Rule of thumb: 1 day = 86,400 sec $\approx 100K$ sec

Simplification: $100M / 100K = 1,000$ QPS per action/day

Key numbers to memorize:

1 million users, 1 req/day $\rightarrow \sim 12$ QPS

10 million users, 10 req/day $\rightarrow \sim 1,200$ QPS

100 million DAU, 1 req/day $\rightarrow \sim 1,200$ QPS

1 billion DAU, 10 req/day $\rightarrow \sim 120,000$ QPS

Interview tip: "Round aggressively. 86,400 $\rightarrow$ 100,000. 1,157 $\rightarrow$ $\sim 1,200$. Interviewers care about order of magnitude, not exact numbers."

Q3. How do you estimate storage requirements?

Pattern:
Storage
estimation

Difficulty: Easy

Instagram-like system:

DAU = 500 million

Photos uploaded per day = 100 million

Average photo size = 200 KB

Daily storage = $100M \times 200$ KB = 20 TB/day

5-year storage = 20 TB $\times 365 \times 5 = 36.5$ PB

With replication (3 $\times$) = ~ 110 PB

Storage for metadata:

Per photo = 500 bytes (URL, userID, timestamp, caption)

Daily metadata = $100M \times 500B = 50$ GB/day

5-year metadata = ~ 90 TB

Size reference:

char = 1 byte

UUID = 16 bytes

int64 = 8 bytes

timestamp = 8 bytes

URL = ~ 100 bytes

tweet = ~ 300 bytes

photo = 100 KB – 5 MB

HD video = 1–2 GB/hour

Interview tip: "Storage is cheap; mention tiered storage: hot (SSD, recent 30 days), warm (HDD, last year), cold (S3 Glacier, older). Dramatically reduces cost."

Q4. How do you estimate bandwidth?

Pattern:
Bandwidth
estimation

Difficulty: Easy

YouTube-like system:

```
DAU = 1 billion
Each user watches: 30 min/day
Video bitrate: 5 Mbps (1080p)

Total watch time = 1B × 30 min = 30 billion min/day
Bandwidth = 30B min × (1/60 hr) × 5 Mbps
            = 2.5 billion Mbps / 86400 sec
            ≈ 28.9 Tbps outbound bandwidth

Upload (user-generated, 500 hours of video/min):
500 hours/min × 60 min/hr × 5 Mbps = 150,000 Mbps = 150 Gbps inbound
```

Reference:

```
1 Gbps  = 125 MB/s
1 Tbps  = 125 GB/s
CDN edge: 100 Gbps per PoP
Single NIC: 10-100 Gbps
```

Interview tip: "Bandwidth drives CDN strategy. If your origin bandwidth exceeds 100 Gbps, you need a CDN with 50+ PoPs globally. State this explicitly."

Q5. What is horizontal vs vertical scaling?

Pattern:
Scaling fund
amentals

Difficulty: Easy

Vertical scaling (Scale Up):

```
Add more CPU/RAM/disk to existing machine
Pros: simple, no code changes, strong consistency
Cons: hardware limits, single point of failure, expensive
Limit: ~128 cores, ~24 TB RAM (AWS u-24tb1.metal)
```

Horizontal scaling (Scale Out):

```
Add more machines, distribute load
Pros: theoretically unlimited, fault tolerant, cheaper commodity hardware
Cons: complexity (distributed state, network, consistency)
Requires: stateless services, distributed data
```

	Vertical	Horizontal
Cost	Exponential	Linear
Complexity	Low	High
Availability	Single PoF	High (with LB)
Limits	Hard ceiling	Nearly infinite
State	Easy (local)	Hard (distributed)

Interview tip: "Database vertical scaling hits limits fast. Horizontal scaling for stateless services is easy (just add servers). Horizontal scaling for stateful services (DB) requires sharding — the hard part."

Q6. What is a load balancer and what algorithms does it use?

Pattern:
Load
balancing

Difficulty: Easy

Load Balancer sits between clients and servers, distributing traffic.

Algorithms:

- Round Robin – requests distributed evenly in sequence
 - Good for: uniform request cost

Layer 4 (Transport): routes by IP/TCP – fast, no content inspection
Layer 7 (Application): routes by HTTP headers, URL, cookies – flexible

Interview tip: "Always mention health checks. A LB without health checks routes to dead servers. Ask about session stickiness — cookie-based is better than IP hash (mobile IPs change)."

Difficulty: Easy

Interview tip: "In production: Nginx or Envoy at the edge (SSL termination, routing), then an internal LB (ALB) for service-to-service. Don't conflate the two."

Difficulty: Easy

Page 5

How it works:

1. User requests `https://cdn.example.com/image.jpg`
2. DNS resolves to nearest CDN PoP (anycast or GeoDNS)
3. PoP checks cache – hit? → serve immediately (10ms)
4. Miss? → PoP fetches from origin, caches, serves (200ms)

Cache strategies:

- Pull CDN: CDN fetches from origin on first miss (lazy)
- Push CDN: You upload content to CDN proactively

Providers: Cloudflare, AWS CloudFront, Fastly, Akamai

Latency: origin = 100-300ms RTT; CDN PoP = 5-20ms RTT

Interview tip: "Always add CDN in your design for any user-facing media. It reduces origin load by 90%+ for static content. Mention cache-control headers for invalidation."

Q9. What is DNS and how does it work in distributed systems?

Pattern:
DNS

Difficulty:
Medium

DNS translates domain names → IP addresses.

Resolution chain:

- Browser cache → OS cache → Recursive resolver →
- Root nameserver → TLD (.com) nameserver → Authoritative nameserver

TTL: how long clients cache DNS records

- Low TTL (30s): fast failover, more DNS queries
- High TTL (24h): fewer queries, slow failover

Techniques:

- GeoDNS: return different IPs based on client location
- Round-robin DNS: return multiple IPs (simple LB, no health checks)
- Anycast: same IP, BGP routes to nearest PoP (used by Cloudflare, 1.1.1.1)
- DNS failover: health checks + automatic IP swap

Records:

- A → IPv4 address
- AAAA → IPv6 address
- CNAME → alias to another domain
- MX → mail server
- TXT → text (verification, SPF, DKIM)
- SRV → service discovery (host:port)

Interview tip: "DNS is not instant failover. With TTL=300s, clients may hit dead servers for 5 minutes. Use health-check-aware DNS (Route53 with health checks) + low TTL for critical services."

Q10. What are the differences between TCP and UDP?

Pattern:
Network fundamentals

Difficulty: Easy

TCP (Transmission Control Protocol):

- Connection-oriented (3-way handshake)
- Reliable delivery (ACK, retransmit)
- Ordered delivery
- Flow control + congestion control
- Higher overhead (~20 byte header + handshake)

Q12. How do you calculate the number of servers needed?Pattern:
Capacity
planning**Difficulty: Easy**

Given:

Peak QPS = 50,000 requests/sec

Avg request duration = 50ms

One server handles = 1000 req/sec (rule of thumb for web server)

Calculation:

$$\begin{aligned}\text{Servers needed} &= \text{Peak QPS} / \text{QPS per server} \\ &= 50,000 / 1,000 = 50 \text{ servers}\end{aligned}$$
With 30% headroom: $50 / 0.7 = \sim 72$ servers

For CPU-bound work:

Concurrency per server = CPU cores $\times$ (1 / CPU utilisation target)16-core server at 70% CPU = ~ 22 concurrent requestsAt 50ms latency: $22 / 0.05s = 440$ QPS/server

For I/O-bound work (waiting on DB):

Concurrency = threads or goroutines waiting

Goroutines: 10,000 goroutines/server at 50ms = 200,000 QPS/server

(Go is extremely efficient for I/O-bound work)

DB connections:

Connection pool per server = 10-25 connections

Total DB connections = 72 servers $\times$ 15 = 1,080 $\rightarrow$ needs connection pooler (PgBouncer)***Interview tip:** "In Go specifically: goroutine-based servers can handle far more concurrent connections than thread-per-request servers (Java/Python). Mention this as an advantage."*

Databases & Storage

Q13. When do you choose SQL vs NoSQL?Pattern:
Database
selection**Difficulty:
Medium**

Choose SQL (PostgreSQL, MySQL) when:

- ACID transactions required (payments, inventory)
- Complex queries (JOINS, aggregations)
- Schema is well-defined and stable
- Data integrity constraints (foreign keys)
- Moderate scale (<10M rows/table without sharding)

Choose NoSQL when:

- Schema-less or frequently changing schema
- Massive scale (billions of records)
- Simple access patterns (key-value, document by ID)
- High write throughput
- Horizontal scaling is required

NoSQL categories:

Key-Value: Redis, DynamoDB – O(1) lookup by key

Document: MongoDB, Firestore – JSON documents, flexible schema
 Wide-Column: Cassandra, HBase – high write throughput, time-series
 Graph: Neo4j, Amazon Neptune – social graphs, fraud detection

Example trade-offs:

User profiles → MongoDB (flexible schema, read by ID)
 Payments → PostgreSQL (ACID, joins, audit)
 Session store → Redis (TTL, fast lookup)
 Activity feed → Cassandra (time-series, append-heavy)
 Social graph → Neo4j (friend-of-friend queries)

Interview tip: "Don't say 'NoSQL scales better.' Say: 'NoSQL sacrifices consistency and query flexibility for horizontal scale. For our access pattern (key lookup by user ID at 1M QPS), DynamoDB is appropriate.'"

Q14. What is database indexing and how does it work?

Pattern:
Indexing

Difficulty:
Medium

Index: auxiliary data structure that speeds up lookups.

Trade-off: faster reads, slower writes (index must be updated).

B-Tree Index (default in PostgreSQL, MySQL):

- Balanced tree, $O(\log N)$ lookup, range scans
- Good for: $=$, $<$, $>$, BETWEEN, ORDER BY
- Size: roughly 10-30% of table size

Hash Index:

- $O(1)$ lookup, no range scans
- Good for: $=$ only

Composite Index: (col_a, col_b)

- Prefix rule: usable for col_a alone, col_a+col_b, NOT col_b alone
- Order matters: put high-cardinality columns first

Covering Index: index includes all columns needed for query

- SELECT name FROM users WHERE email=?
- Index on (email, name) → no table fetch needed

Partial Index: index on subset of rows

- CREATE INDEX ON orders(user_id) WHERE status='active'
- Smaller index, faster for filtered queries

Full-Text Index: inverted index for text search

- PostgreSQL: tsvector / tsquery
- Good for: LIKE '%term%' alternatives

Interview tip: "EXPLAIN ANALYZE in PostgreSQL shows query plan and actual execution time. Always run it before claiming a query is slow. Seq scan = missing index. Index scan = good."

Q15. What is database replication and what are its types?

Pattern:
Replication

Difficulty:
Medium

Replication: copying data to multiple nodes for redundancy and read scaling.

Single-Leader (Master-Replica):

One writable primary → changes replicated to read replicas
 Pros: simple, read scaling, failover
 Cons: write bottleneck at primary, replication lag

Replication lag: replica may be seconds/minutes behind primary
 Read-your-writes: after write to primary, read from primary

Multi-Leader:

Multiple writable nodes, sync changes peer-to-peer
 Pros: write throughput, multi-datacenter
 Cons: write conflicts (last-write-wins, merge, CRDTs)
 Used by: CockroachDB, Google Docs (CRDT)

Leaderless (Dynamo-style):

Writes go to W out of N nodes
 Reads from R out of N nodes
 Quorum: $W + R > N$ ensures overlap
 Used by: Cassandra, DynamoDB

Synchronous replication: primary waits for replica ACK

- Strong consistency, higher write latency

Asynchronous replication: primary returns before replica ACK

- Lower write latency, possible data loss on primary failure

PostgreSQL streaming replication: async by default, can be sync

Interview tip: "Always ask: 'Is replication lag acceptable?' For user profile reads: yes. For inventory (race condition on last item): no — read from primary or use synchronous replication."

Q16. What is database sharding?

Pattern:
Sharding

Difficulty: Hard

Sharding: horizontal partitioning — split data across multiple DB nodes.

Shard Key selection (most important decision):

- Good: high cardinality, even distribution, no hotspots
- Bad: low cardinality (country = 3 values), monotonically increasing (time-based → hotspot)

Sharding strategies:

Range-based: shard by user_id range [0-1M, 1M-2M, ...]
 Pros: range queries easy
 Cons: hotspots (latest data all on one shard)

Hash-based: shard = hash(user_id) % N
 Pros: even distribution
 Cons: range queries require all shards, resharding hard

Directory-based: lookup table maps keys → shards
 Pros: flexible, easy resharding
 Cons: lookup table is a bottleneck/SPOF

Consistent Hashing: virtual nodes on a ring
 Pros: minimal data movement when adding/removing shards
 Cons: more complex

Challenges:

- Cross-shard joins: not supported → denormalise or application-level join
- Cross-shard transactions: use 2-phase commit or saga pattern
- Resharding: double-write during migration, painful
- Hot shards: add virtual nodes, split shard

Rule of thumb: start with 1 shard, shard when >50M rows and tuning is exhausted

Interview tip: "Interviewers want to hear: shard key selection rationale, resharding strategy, and how you handle cross-shard queries. Most systems shard too early — say 'I'd exhaust vertical scaling first.'"

Q17. What is the CAP theorem?

Pattern:
Distributed
systems
theory

Difficulty:
Medium

CAP Theorem: In a distributed system, you can guarantee at most 2 of:

- C – Consistency: every read sees the most recent write
- A – Availability: every request gets a response (not an error)
- P – Partition Tolerance: system works despite network partitions

Network partitions ALWAYS happen in distributed systems.
Therefore real choice is: C vs A during a partition.

CP systems (consistency over availability):

- During partition: reject requests rather than serve stale data
- Examples: HBase, Zookeeper, Spanner, etcd, Redis (single-node)
- Use: financial transactions, inventory

AP systems (availability over consistency):

- During partition: serve possibly stale data
- Examples: DynamoDB, Cassandra, CouchDB
- Use: social feeds, user profiles, DNS

CA systems: impossible in distributed systems (can't tolerate partition)

- Only applies to single-node or non-distributed systems

Modern nuance – PACELC extends CAP:

- Even without partition (P):
- Latency (L) vs Consistency (C) trade-off
- Examples: DynamoDB (EL – low latency, eventual consistency)
- Spanner (EC – high consistency, higher latency)

Interview tip: "CAP is often misunderstood. The right answer: 'All distributed systems must tolerate partitions (P). The real question is CP vs AP.' Then map your design to one."

Q18. What is ACID vs BASE?

Pattern:
Consistency
models

Difficulty:
Medium

ACID (relational databases):

- Atomicity: transaction is all-or-nothing
- Consistency: DB moves from one valid state to another
- Isolation: concurrent transactions don't interfere
- Durability: committed data persists (written to disk/WAL)

Cost: locks, 2-phase commit, WAL overhead → lower throughput

BASE (NoSQL, distributed):

- Basically Available: system is available even with failures
- Soft state: state may change over time (even without input)
- Eventually Consistent: system will be consistent... eventually

Benefit: higher throughput, lower latency, better availability

Isolation Levels (SQL):

Read Uncommitted: see dirty reads (dangerous)
 Read Committed: no dirty reads (PostgreSQL default)
 Repeatable Read: no non-repeatable reads
 Serializable: transactions run as if sequential (safest, slowest)

Anomalies:

Dirty read: reading uncommitted writes
 Non-repeatable read: re-read row changes within transaction
 Phantom read: range query returns different rows
 Write skew: two txns read same data, make conflicting updates

Interview tip: "For payment systems: ACID with Serializable isolation. For social feed: BASE with eventual consistency. Know where your system falls on this spectrum."

Q19. What is a write-ahead log (WAL) and why is it important?

Pattern:
Durability

Difficulty:
Medium

WAL: append-only log of every change before applying to data files.
 Purpose: crash recovery – replay WAL to restore state after failure.

How it works:

1. Transaction begins
2. Changes written to WAL (sequential write = fast)
3. Client gets ACK – data is durable
4. DB flushes WAL to data files asynchronously (background)
5. On crash: replay WAL from last checkpoint

Benefits:

- Durability without immediate random writes to data files
- Sequential writes are 10-100× faster than random writes
- Replication: stream WAL to replicas (PostgreSQL streaming replication)
- Point-in-time recovery: replay WAL to any timestamp

WAL in different systems:

PostgreSQL: WAL (pg_wal/)
 MySQL/InnoDB: redo log
 Kafka: commit log (IS the WAL)
 RocksDB/LevelDB: Write-Ahead Log before memtable
 Redis AOF: Append-Only File (WAL-like persistence)

fsync: force OS buffer to disk

fsync=on: safe but slower (disk write per transaction)
 fsync=off: dangerous – data loss on power failure

Interview tip: "PostgreSQL's WAL enables logical replication, streaming replication, and PITR. Understanding WAL shows you know databases beyond SQL syntax."

Q20. What is the difference between B-Tree and LSM-Tree storage engines?

Pattern:
Storage engine
internals

Difficulty: Hard

B-Tree (PostgreSQL, MySQL InnoDB):

Structure: sorted balanced tree on disk
 Write: find page, update in-place → random I/O
 Read: $O(\log N)$ traversal → fast reads

```
LSM-Tree (Cassandra, RocksDB, LevelDB):
Structure: in-memory buffer (memtable) → SSTables on disk
Write: append to WAL + memtable → sequential I/O → flush to SSTable
Read: check memtable + each SSTable level (bloom filter helps)
Compaction: merge SSTables periodically (background I/O)
Good for: write-heavy workloads, time-series, event streams
```

B-Tree		LSM-Tree
Writes	Random I/O (slow)	Sequential (fast)
Reads	Fast	Slower (multiple files)
Space	Less wasted	Write amplification
Compaction	None needed	Required (background)
Use case	OLTP	Write-heavy, analytics

Interview tip: "LSM trees are why Cassandra and RocksDB handle millions of writes/sec. The memtable absorbs all writes in RAM, then flushes sequentially. Random write → sequential write transformation."

Pattern:
Connection
managemen
t

Difficulty: Easy

Without pooling:
100 req/sec × 5ms connection time = 500ms wasted per second in connection overhead

```
With pooling (PgBouncer for PostgreSQL):
  Pool of 20 persistent connections shared by 1000 goroutines
  Connection reuse → ~0ms overhead after warmup
```

```
Parameters:
  min_pool_size: connections always kept open (warm pool)
  max_pool_size: maximum concurrent connections
  connection_lifetime: recycle old connections (prevent stale)
  idle_timeout: close unused connections
```

```
PgBouncer modes:
  Session pooling: 1 DB conn per client session (least efficient)
  Transaction pooling: 1 DB conn per transaction (most efficient, default)
  Statement pooling: 1 DB conn per statement (limits prepared statements)
```

```
Rules of thumb:
  Max connections per PostgreSQL: 100-500
  With PgBouncer: 1000s of app connections → 100 DB connections
  app_servers × threads_per_server × max_pool = total DB connections
```

```
Go specifics:
database/sql has built-in pool
db.SetMaxOpenConns(25)
db.SetMaxIdleConns(10)
db.SetConnMaxLifetime(5 * time.Minute)
```

Interview tip: "Mention PgBouncer by name. Not having a connection pooler with PostgreSQL at scale is a common mistake. At 1000 connections, PostgreSQL itself becomes a bottleneck."

Q22. What is database denormalization and when do you use it?

Pattern:
Data
modelling

Difficulty:
Medium

Normalization: eliminate redundancy, ensure data integrity (OLTP)

Denormalization: introduce redundancy to speed up reads (OLAP/high-scale)

Example – Twitter timeline:

Normalized:

tweets table: tweet_id, user_id, content, created_at

followers table: follower_id, followed_id

```
Query timeline: SELECT t.* FROM tweets t
                JOIN followers f ON t.user_id = f.followed_id
                WHERE f.follower_id = ? ORDER BY t.created_at DESC
```

Problem: JOIN across billions of rows = too slow

Denormalized (fan-out on write):

timeline table: user_id, tweet_id, content, author_name, author_avatar

Pre-compute timeline for each user on tweet creation

```
Query timeline: SELECT * FROM timeline WHERE user_id=? ORDER BY created_at DESC
```

Fast: single table scan with index

Trade-offs:

Denormalized reads: $O(1)$ → blazing fast

Write amplification: 1 tweet × 10K followers = 10K writes

Storage: duplicated data

Consistency: must update all copies on profile change

When to denormalize:

- Read QPS >> Write QPS
- Join query is the bottleneck
- Data rarely changes (e.g., tweet content is immutable)

Interview tip: "Twitter's timeline is the canonical denormalization example. But don't denormalize everything — start normalised, then denormalize specific hot query paths."

Q23. What is the N+1 query problem and how do you fix it?

Pattern:
Query
optimisation

Difficulty:
Medium

N+1 problem: fetching N records then querying 1 more time per record.

Example (Go + SQL):

```
posts, _ := db.Query("SELECT id FROM posts LIMIT 100") // 1 query
for _, post := range posts {
    author, _ := db.QueryRow("SELECT name FROM users WHERE id=?", post.UserID) // 100 queries!
}
// Total: 101 queries
```

Fix 1: JOIN

```
SELECT p.*, u.name FROM posts p
JOIN users u ON p.user_id = u.id
LIMIT 100
// 1 query
```

Fix 2: IN clause batching

```

userIDs := extractUserIDs(posts)
users, _ := db.Query("SELECT id, name FROM users WHERE id IN (?)", userIDs)
userMap := toMap(users)
for _, post := range posts { post.Author = userMap[post.UserID] }
// 2 queries

```

Fix 3: DataLoader (batch + cache, common in GraphQL)

Batch all user lookups within a request window
 Cache results within request lifecycle

Fix 4: Preload / Eager loading (ORMs)

```
db.Preload("Author").Find(&posts) // GORM generates JOIN automatically
```

Interview tip: "N+1 is the most common performance bug in production systems. Detection: log queries with duration, alert on >10 queries for a single request. Fix during code review."

Q24. What is OLTP vs OLAP?

Pattern:
Workload
types

Difficulty: Easy

OLTP (Online Transaction Processing):

Many small transactions, real-time, read/write
 Highly normalized, ACID, low latency
 Example: e-commerce checkout, banking, social feed
 Systems: PostgreSQL, MySQL, DynamoDB

OLAP (Online Analytical Processing):

Few large queries, batch/near-real-time, mostly read
 Denormalized (star schema), column-oriented, high throughput
 Example: business intelligence, reporting, dashboards
 Systems: BigQuery, Redshift, Snowflake, ClickHouse

Column-oriented storage (OLAP):

Data stored column by column: [col1_all_rows], [col2_all_rows]
 Query: SELECT SUM(revenue) FROM orders → reads ONLY revenue column
 Compression: same data type → 10-100× compression
 vs Row-oriented: reads entire row to access one column → wasteful

Lambda architecture:

Batch layer (Hadoop/Spark): accurate, slow (hours)
 Speed layer (Kafka/Flink): fast, approximate (seconds)
 Serving layer: merges both for queries

Kappa architecture: stream-only (Kafka as source of truth)

Simpler, stream everything, replay for reprocessing

Data warehouse pipeline:

OLTP DB → Kafka/CDC → ETL (dbt) → Data Warehouse → BI tools

Interview tip: "When someone asks 'how would you build an analytics dashboard?' — separate OLAP store from OLTP DB immediately. Never run analytics queries on your production OLTP database."

Q25. What is a distributed transaction and how do you handle it?

Pattern:
Distributed
transactions

Difficulty: Hard

Problem: updating data across multiple services/databases atomically.

Example: Transfer \$100: debit from Account Service + credit to Payment Service

Option 1: Two-Phase Commit (2PC)

Phase 1 (Prepare): coordinator asks all participants to prepare

Phase 2 (Commit): if all say yes → commit; else → rollback

Pros: strong consistency

Cons: blocking, coordinator is SPOF, latency, slow

Option 2: Saga Pattern (preferred for microservices)

Break transaction into local transactions + compensating transactions

Each service does its part; on failure, run compensating actions

Choreography saga (event-driven):

OrderService → emits OrderCreated

PaymentService → listens, charges, emits PaymentCharged

InventoryService → listens, reserves stock, emits StockReserved

On failure: each service emits failure event → others compensate

Orchestration saga:

SagaOrchestrator drives each step, handles failures explicitly

Clearer failure handling, easier to debug

Option 3: Outbox Pattern

Write to DB + outbox table in same transaction

Outbox poller reads and publishes events to Kafka

Ensures at-least-once delivery without distributed transaction

Used with Saga or event-driven architectures

Option 4: CRDT / Eventual Consistency

Accept eventual consistency for non-critical data

Interview tip: "2PC is rarely used in modern microservices — too fragile. Saga pattern with the Outbox Pattern is the production standard. Mention both."

Caching & CDN

Q26. What are the caching strategies?

Pattern:
Cache
strategies

Difficulty:
Medium

Cache-Aside (Lazy Loading):

App checks cache → miss → fetch from DB → populate cache → return

Pros: cache only what's needed, resilient to cache failure

Cons: cache miss = 3 trips (cache + DB + cache write), stale data

Read-Through:

Cache sits in front, fetches from DB on miss automatically

Pros: transparent to app

Cons: cold start (every key misses once)

Write-Through:

Write to cache + DB synchronously

Pros: always fresh cache

Cons: write latency (waits for both), cache pollution (write things never read)

Write-Behind (Write-Back):

Write to cache, async flush to DB

Pros: very fast writes

Cons: data loss risk (cache crash before flush)

Write-Around:

Write directly to DB, skip cache

Pros: avoids caching data written once and never read again

Cons: cold reads after write

Refresh-Ahead:

Proactively refresh cache before expiry based on predicted access

Pros: no cold misses for popular data

Cons: complex, may refresh data never read again

Production pattern (most common):

Read: Cache-Aside

Write: Write-Around or Write-Through with short TTL

Invalidation: delete on write (not update) + TTL as safety net

Interview tip: "The most asked question is cache invalidation. Best answer: 'Delete on write + TTL. Never update cached objects (causes race conditions). Short TTL as a safety net.'"

Q27. What is Redis and what are its data structures?

Pattern:
Redis fundamentals

Difficulty: Easy

Redis: in-memory data structure store. Single-threaded (mostly).

Persistence: RDB (snapshot) or AOF (append-only log) or both.

Replication: primary-replica. Cluster: hash-slot-based sharding.

Data Structures:

String: SET/GET. Use: counters, cache any value, sessions

SET user:1:name "Alice"

INCR page:views:home

List: LPUSH/RPUSH/LPOP. Use: queues, recent activity

LPUSH recent:user:1 "item_id_42"

LRANGE recent:user:1 0 9 → last 10 items

Hash: HSET/HGET. Use: object fields, user profile

HSET user:1 name Alice age 30 email alice@example.com

HGETALL user:1

Set: SADD/SMEMBERS/SINTER. Use: unique visitors, tags, friends

SADD user:1:friends "2" "3" "4"

SINTER user:1:friends user:2:friends → mutual friends

Sorted Set: ZADD/ZRANGE/ZRANK. Use: leaderboards, rate limiting, timelines

ZADD leaderboard 1500 "alice" 1200 "bob"

ZREVRANGE leaderboard 0 9 → top 10

HyperLogLog: approximate unique count. Use: unique visitor count

PFADD visitors:today "user1" "user2"

PFCOUNT visitors:today → ~unique count, 0.81% error

Pub/Sub: publish-subscribe. Use: real-time notifications, chat
Streams: append-only log. Use: event sourcing, audit log
Lua scripts: atomic multi-command operations

Interview tip: "Redis Sorted Sets are the most interview-asked structure. Know: ZADD $O(\log N)$, ZRANK $O(\log N)$, ZRANGE $O(\log N + M)$. Used for leaderboards, rate limiters, delayed job queues."

Q28. What is cache eviction policy?

Pattern:
Cache man
agement

Difficulty: Easy

When cache is full, eviction policy decides what to remove.

LRU (Least Recently Used): evict item not accessed for longest time
Best for: recently accessed data is likely to be re-accessed soon
Implementation: hash map + doubly-linked list ($O(1)$ operations)
Redis: allkeys-lru or volatile-lru

LFU (Least Frequently Used): evict item accessed fewest times
Best for: some items are accessed less but recently (new items)
LFU can evict a "new" popular item – needs decay mechanism
Redis: allkeys-lfu or volatile-lfu

FIFO: evict oldest item regardless of access
Simple but ignores access patterns

Random: randomly evict any item
Surprisingly competitive with LRU in some workloads

TTL-based: evict expired items (not really an eviction policy)
Always used in combination with above policies

Redis eviction policies:
noeviction: return error when full (bad for cache, ok for queue)
allkeys-lru: LRU across all keys (most common for pure cache)
volatile-lru: LRU among keys with TTL set
allkeys-random: random eviction
volatile-ttl: evict key with shortest TTL first

Rule: allkeys-lru is best for general caching.
volatile-lru when you have mixed use (cache + queue in same Redis).

Interview tip: "LRU is $O(1)$ with the right implementation (doubly linked list + hash map). A naive LRU that scans the cache is $O(N)$. Know the implementation from the LLD section."

Q29. What is a cache stampede and how do you prevent it?

Pattern:
Cache
reliability

Difficulty: Hard

Cache stampede (thundering herd):
Popular cache key expires
100s of requests simultaneously miss cache
All hit DB simultaneously → DB overload → slow responses → more misses

Prevention strategies:

1. Mutex/Lock (single-flight):

First goroutine gets lock → fetches from DB → populates cache
Others wait for lock → get result from cache
Go: golang.org/x/sync/singleflight
Cost: adds latency for waiting requests

2. Probabilistic Early Expiration (PER):
Refresh cache slightly before TTL expires with some probability
P(refresh) increases as expiry approaches
Key insight: no synchronization needed, just background refresh
3. Cache-warmup on deploy:
Pre-populate cache before TTL expires
Background job refreshes popular keys
Uses access logs to predict what to warm
4. Jittered TTL:
 $TTL = base_ttl + random(0, jitter_amount)$
Prevents many keys expiring at the same time
Popular items: $TTL=5min \pm 30sec$
Simple and effective
5. Stale-while-revalidate:
Serve stale data immediately
Trigger async background refresh
Client sees slightly old data but no stampede

Production: combine singleflight + jittered TTL + background refresh

Interview tip: "singleflight is the Go idiom. In other languages: Redis SETNX as a distributed lock. Jittered TTL is the simplest prevention — always add it."

Q30. What is a write-through vs write-around cache?

Pattern:
Write
strategies

Difficulty: Easy

Write-Through:

Write → cache AND DB simultaneously (synchronous)
Cache always consistent with DB
Every written item is cached (even if never read again)

Pros: cache hit on first read after write
Cons: write latency (waits for both), cache pollution
Use: read-heavy with frequent re-reads of written data

Write-Around:

Write → DB only (bypass cache)
Cache loaded lazily on first read (cache-aside pattern)

Pros: no cache pollution, no write latency penalty
Cons: read after write always misses cache (cold read)
Use: data written once and rarely read (logs, audit trails)

Write-Behind (Write-Back):

Write → cache only (async DB flush)

Pros: very fast writes, absorbs write bursts
Cons: data loss if cache fails before flush

Use: session data, shopping cart (acceptable to lose)
Mitigation: Redis AOF persistence + replica

Combined pattern:

Write: Write-Around (write direct to DB, delete/invalidate cache)
Read: Cache-Aside (check cache, miss → DB → populate cache)
Result: cache populated on demand, no stale data, no pollution

Interview tip: "The combined Write-Around + Cache-Aside is the safest production pattern. On write: delete cache key (not update). Cache is eventually populated when someone reads."

Q31. What is Redis Cluster and how does it shard data?

Pattern:
Redis
scaling

Difficulty: Hard

Redis Cluster: horizontal scaling via hash slot sharding.
Total hash slots: 16,384

Key → $\text{CRC16}(\text{key}) \% 16384$ → hash slot → node

3-node cluster: Node A: 0-5461, Node B: 5462-10922, Node C: 10923-16383

Hash tags: {user_id} → hash only user_id
Forces related keys to same slot
Example: {user:42}:profile and {user:42}:sessions
→ same slot → MULTI/EXEC transactions work

Scaling:

Add node: Redis rebalances slots automatically
Remove node: drain slots first, then remove

Replication in cluster:

Each primary has 1+ replicas
6 nodes = 3 primaries + 3 replicas (recommended minimum)

Limitations:

Multi-key operations: only if all keys in same slot (use hash tags)
Lua scripts: all keys must be in same slot
Pub/Sub: works across cluster
No cross-slot transactions

When to use Redis Cluster vs single Redis:

Single Redis: up to ~100GB RAM, ~1M ops/sec (fast enough for most)
Redis Cluster: >100GB data or need >1M ops/sec
Many teams delay clustering too long – Redis Cluster adds complexity

Alternative: Redis Sentinel (HA without sharding)

Primary + replicas + Sentinel processes for automatic failover
Simpler than Cluster, no sharding

Interview tip: "Know hash slots = 16,384. Know hash tags for co-location. Redis Cluster is overkill for most SDE2 designs — mention Sentinel for HA first, Cluster only for massive scale."

Q32. When do you cache at the application vs database vs CDN level?

Pattern:
Cache
placement

Difficulty:
Medium

CDN cache (edge, global):

What: static assets, public API responses, HTML pages

TTL: minutes to days

Benefits: lowest latency (20ms vs 200ms), offload origin by 90%+

Cache-Control: public, max-age=3600

Invalidation: cache key = URL + version (bust with query param)

Application-level cache (in-process):

What: computed results, parsed configs, user session

Where: memory inside the process (e.g., sync.Map in Go)

TTL: seconds to minutes

Benefits: 0ms (no network), perfect for hot lookup data

Limits: not shared across instances, lost on restart

Distributed cache (Redis):

What: user sessions, rate limit counters, expensive DB query results

Where: shared across all app servers

TTL: seconds to hours

Benefits: shared state, survives app restart, rich data structures

Latency: ~1ms (loopback) to ~5ms (cross-AZ)

Database-level cache (query cache, buffer pool):

What: frequently accessed table pages, index pages

Where: PostgreSQL shared_buffers (RAM)

TTL: managed by DB engine

Benefits: automatic, no code changes

Set shared_buffers = 25-40% of total RAM

Hierarchy: CDN → App cache → Redis → DB buffer pool → DB disk

Each layer handles a subset of traffic, reducing pressure on the next.

Interview tip: "Design cache layers from outer to inner. CDN should absorb 90% of static traffic. Redis absorbs 80% of DB reads. DB only sees 10% of total requests. State these percentages."

Q33. How does cache invalidation work across multiple services?

Pattern:
Distributed
cache
invalidation

Difficulty: Hard

Problem: Service A writes to DB. Service B has cached the same data.

How does Service B know to invalidate its cache?

Strategy 1: TTL only (simplest)

Every cache entry has a TTL (e.g., 5 minutes)

Stale data serves until TTL expires

Good enough for: user profiles, product catalog

Not OK for: inventory, prices, balances

Strategy 2: Event-driven invalidation

Writer publishes "cache:invalidate:user:42" to message broker

All services listening delete their local cache entry

Challenge: at-least-once delivery → idempotent invalidation

Implementation: Redis Pub/Sub or Kafka topic

Strategy 3: Write-Through invalidation

Any write to DB triggers cache delete in the same service

Other services rely on TTL or subscribe to events

Strategy 4: Versioned cache keys

Key = "user:42:v7" where v7 is the current version
 On update: increment version in DB, new key on next read
 Old keys expire via TTL
 No explicit invalidation needed
 Trade-off: old versions hang around until TTL

Strategy 5: CDC (Change Data Capture) - Debezium

Debezium reads PostgreSQL WAL stream
 Publishes row changes to Kafka
 Cache invalidation service consumes Kafka → deletes Redis keys
 Fully decoupled, near-real-time

Interview tip: "The best production pattern: Delete-on-write (in same transaction as DB write) + short TTL (safety net) + event-driven invalidation for cross-service. Don't update cached values — always delete."

Q34. What is a Bloom filter and where is it used in caching?

Pattern:
 Probabilistic
 data
 structures

Difficulty: Hard

Bloom filter: space-efficient probabilistic set membership test.

"Definitely not in set" or "Probably in set" (false positives possible)
 No false negatives.

How it works:

- Bit array of size M, initialised to 0
- K hash functions
- Add element: set bits at k positions to 1
- Check: if all k positions are 1 → probably in set
- False positive rate $\approx (1 - e^{(-kn/m)})^k$

Properties:

No deletions (use Counting Bloom Filter for deletions)
 False positive rate decreases with larger M
 $O(k)$ insert and lookup (k = number of hash functions)

Applications in caching:

Cache negative results: "Does user_id 99999 exist?"
 Without bloom filter: cache miss → DB query → "not found" → cache {99999: null}
 With bloom filter: check bloom filter → "definitely not" → skip DB entirely
 Saves DB hit for non-existent keys

Real-world uses:

Cassandra: bloom filter per SSTable to avoid reading wrong SSTable
 HBase: bloom filter on each HFile
 Redis: RedisBloom module
 Chrome: malicious URL detection (downloaded bloom filter)
 Akamai CDN: avoid caching one-time-access content

Space vs accuracy:

1% false positive rate: ~9.6 bits per element
 0.1% false positive rate: ~14.4 bits per element
 1M elements at 1% FPR: ~1.2 MB

Interview tip: "Bloom filters are asked in almost every big-tech HLD interview. Know: no false negatives, can have false positives, cannot delete (standard), $O(1)$ lookup. Use case: avoid DB lookup for non-existent keys."

Messaging & Queues

Q35. What is a message queue and when do you use one?

Pattern:
Async
messaging

Difficulty: Easy

Message queue: decouples producer from consumer asynchronously.

When to use:

- Decouple services (payment → email notification)
- Absorb traffic spikes (queue requests, process at steady rate)
- Async processing (video encoding, email sending)
- Fan-out (one event → multiple consumers)
- Retry logic (failed messages retried automatically)
- Rate limiting consumers (prevent overwhelming downstream)

Message delivery semantics:

- At-most-once: message may be lost, never duplicated (fire-and-forget)
- At-least-once: message may be duplicated, never lost (ack after processing)
- Exactly-once: no loss, no duplicates (hardest, most expensive)
- Idempotent consumers + at-least-once = effectively exactly-once

Pull vs Push:

- Pull (Kafka, SQS): consumer polls queue
 - Pros: consumer controls rate, no overwhelming
- Push (RabbitMQ, SNS): broker pushes to consumer
 - Pros: lower latency, simpler consumer
 - Cons: consumer can be overwhelmed

Queue vs Topic:

- Queue: one consumer per message (point-to-point)
- Topic: all subscribers get every message (pub/sub)
- Kafka: topic with consumer groups (each group gets all messages; one member per group per partition)

Interview tip: "Always ask: 'Do consumers need to process independently or together?' If each notification should go to multiple services, use a topic (Kafka/SNS). Point-to-point → SQS/RabbitMQ queue."

Q36. What is Apache Kafka and how does it work?

Pattern:
Kafka
internals

Difficulty: Hard

Kafka: distributed append-only log. High-throughput, durable, replayable.

Core concepts:

- Topic: named log (like a table)
- Partition: topic split into ordered, immutable log segments
 - key → hash → partition (consistent routing)
- Offset: message position within a partition
- Consumer Group: multiple consumers, each partition read by ONE consumer
- Broker: Kafka server (one or more per cluster)
- Replication: each partition has 1 leader + N-1 followers

Write path:

Page 24

When to choose:

"I need to send an email after payment" → RabbitMQ/SQS (simple task queue)

"I need multiple services to react to payment + replay + analytics" → Kafka

Interview tip: "RabbitMQ deletes messages after consumption. Kafka retains them. This is the fundamental difference — Kafka is a log, RabbitMQ is a queue. Replay = Kafka."

Q38. What is the outbox pattern?

Pattern: Transactional outbox

Difficulty: Hard

Problem: atomic write to DB + publish to Kafka — two separate systems.

If DB commit succeeds but Kafka publish fails → data inconsistency

```
// WRONG — not atomic
db.Exec("INSERT INTO orders ...") // may succeed
kafka.Publish("order.created", ...) // may fail → event lost
```

Outbox Pattern:

Write to DB + outbox table in ONE transaction

Separate process reads outbox → publishes to Kafka → marks as processed

```
// CORRECT
tx.Exec("INSERT INTO orders ...")
tx.Exec("INSERT INTO outbox (event_type, payload) VALUES (...)")
tx.Commit() // atomic — both or neither

// Outbox poller (separate goroutine/process)
rows := db.Query("SELECT * FROM outbox WHERE processed=false ORDER BY id")
for _, row := range rows {
    kafka.Publish(row.EventType, row.Payload)
    db.Exec("UPDATE outbox SET processed=true WHERE id=?", row.ID)
}
```

Implementations:

Polling: simple, slight delay

Debezium (CDC): reads PostgreSQL WAL → publishes to Kafka (near-real-time, no polling)

Transactional Outbox guarantees:

At-least-once delivery (idempotent consumers required)

No message loss

No phantom events (events only for committed transactions)

Interview tip: "The outbox pattern is the standard solution for 'write to DB and publish to Kafka atomically.' Every senior Go backend engineer should know this pattern cold."

Q39. What is backpressure in a messaging system?

Pattern: Flow control

Difficulty: Medium

Backpressure: mechanism to prevent fast producers from overwhelming slow consumers.

Without backpressure:

Producer: 10,000 msg/sec

Consumer: 1,000 msg/sec

Queue grows 9,000 msg/sec → OOM → crash

With backpressure:

Consumer signals capacity to producer
 Producer slows down or drops (fail-fast)

Strategies:

1. Bounded queue: queue has max size; producer blocks when full
 Pro: simple
 Con: producer blocks (cascades back)
2. Drop: discard messages when queue is full
 Pro: producer never blocks
 Con: data loss acceptable? (logs: yes; payments: no)
3. Rate limiting: limit producer to consumer's throughput
 Pro: no loss, no blocking
 Con: requires capacity estimation
4. Consumer groups + partition scaling (Kafka):
 Add more consumer instances → more parallelism
 Auto-scaling: CloudWatch alarm on consumer lag → add instances
5. Circuit breaker: stop producing if consumer is unhealthy

Kafka consumer lag:

Metric: consumer group lag = latest_offset - consumer_committed_offset
 Alert threshold: lag > 10,000 → consumer can't keep up
 Action: scale consumers (add instances or increase partitions)

Go implementation:

Bounded channel = backpressure
 make(chan Job, 100) → producer blocks at 100
 make(chan Job, 0) → unbuffered, perfect synchronisation

Interview tip: "Consumer lag is the key Kafka metric to monitor. Alert on lag > threshold → auto-scale consumers. Discuss this when asked about Kafka reliability."

Q40. What is event-driven architecture?

Pattern:
EDA

Difficulty:
Medium

Event-driven architecture: services communicate via events (not direct calls).

Components:

Event producer: emits event when something happens
 Event broker: Kafka, SNS, EventBridge
 Event consumer: reacts to events asynchronously

Benefits:

Loose coupling: producer doesn't know about consumers
 Extensibility: add consumer without changing producer
 Resilience: consumer failure doesn't affect producer
 Audit trail: events are immutable facts (event log = audit log)

Patterns:

Event Notification: "order.placed" → email consumer reacts
 Event-Carried State Transfer: event contains full object data → no need to call producer's API
 Event Sourcing: events are the source of truth → derive state by replaying

Challenges:

Eventual consistency: consumers may lag
 Idempotency: events may be delivered twice → consumers must handle
 Ordering: Kafka guarantees order per partition only
 Schema evolution: changing event schema breaks consumers
 Use: Avro with Schema Registry or JSON with versioned fields

Commands vs Events:

Command: "CreateOrder" – imperative, one receiver, expects response
 Event: "OrderCreated" – past tense, broadcast, multiple receivers

Choreography vs Orchestration:

Choreography: services react to events independently (loose coupling)
 Orchestration: central coordinator directs services (easier to trace)

Interview tip: "Events should be in past tense ('OrderCreated', 'PaymentFailed'). Commands are imperative ('CreateOrder'). Getting the semantics right shows design maturity."

Q41. How do you handle duplicate messages (idempotency)?

Pattern: Idempotency

Difficulty: Medium

At-least-once delivery means duplicates happen. Consumers must be idempotent.

Strategies:

1. Natural idempotency:

"Set user.email = X" → running twice has same result
 "Set order status to SHIPPED" → idempotent

2. Idempotency key (database):

Each message has unique ID (Kafka offset or UUID)
 Before processing: INSERT INTO processed_messages (msg_id) ON CONFLICT DO NOTHING
 If insert succeeds → process
 If insert fails (conflict) → already processed, skip

3. Conditional update:

UPDATE orders SET status='shipped'
 WHERE order_id=? AND status='processing'
 → Only updates if in expected state; noop if already shipped

4. Redis deduplication:

SET processed:{msg_id} 1 EX 86400 NX
 If SET succeeds → process
 If SET fails (key exists) → duplicate, skip

5. Event sourcing (naturally idempotent):

Each event has sequence number
 "Apply event 42 to aggregate" → if aggregate already at version 42+ → skip

Implementation in Go:

```

func (h *Handler) HandlePaymentProcessed(msg kafka.Message) error {
    key := fmt.Sprintf("processed:%s", msg.Key)
    if !redis.SetNX(key, 1, 24*time.Hour) {
        return nil // duplicate – skip
    }
    return h.processPayment(msg.Value)
}
  
```

Interview tip: "Idempotency is non-negotiable for payment processing. Always ask: 'What happens if this message is delivered twice?' If the answer is 'bad things,' you need idempotency."

Q42. What is a dead letter queue (DLQ)?

Pattern:
Error
handling in
queues

Difficulty: Easy

Dead Letter Queue: separate queue for messages that fail processing.

When a message goes to DLQ:

- Exceeded max retry attempts
- Message could not be deserialized
- Consumer threw unretryable exception

Why DLQ is important:

Without DLQ: failed messages block queue (head-of-line blocking) or are dropped

With DLQ: bad messages isolated → main queue keeps flowing → alerts for investigation

DLQ workflow:

1. Consumer fails to process message
2. Message retried N times (exponential backoff)
3. After N failures → moved to DLQ
4. Alert triggered (PagerDuty/Slack)
5. Engineer investigates → fix bug
6. Replay messages from DLQ to main queue

Implementation (AWS SQS):

Main queue → maxReceiveCount=5 → DLQ

CloudWatch alarm: DLQ depth > 0 → alert

Implementation (Kafka):

No built-in DLQ

Consumer catches exception → manually publish to "orders.dlq" topic

Consumer group on DLQ topic for investigation/replay

Monitoring:

Alert on DLQ depth > 0 (immediate investigation)

DLQ depth trend (growing = systemic issue)

Retry policy:

Immediate retry: transient network error

Delayed retry (backoff): service temporarily unavailable

No retry (→ DLQ): bad message format, business rule violation

Interview tip: "Every queue in production needs a DLQ. Period. Not having one means failed messages are either dropped (data loss) or block the queue (availability issue)."

Q43. What is CQRS (Command Query Responsibility Segregation)?

Pattern:
CQRS

Difficulty: Hard

CQRS: separate read model from write model.

Traditional (CRUD):

Single model for reads and writes → compromises for both

CQRS:

Write side (Command): handles mutations, domain logic, writes to write DB
Read side (Query): handles queries, optimised read model, can be different DB
Sync: events propagate writes to read model asynchronously

Example — Order System:

Write model (PostgreSQL, normalised):

orders: (id, user_id, status, created_at)
order_items: (order_id, product_id, qty, price)

Read model (Elasticsearch or DynamoDB, denormalised):

order_summary: (order_id, user_name, item_names, total, status, created_at)
→ Optimised for dashboard query "show me my orders"

Sync: OrderCreated event → read model projector → upsert into Elasticsearch

Benefits:

Read model optimised exactly for query pattern
Independent scaling of reads and writes
Different storage for different needs

Challenges:

Eventual consistency: read model lags behind write model
Complexity: two models to maintain
More moving parts (event bus, projections)

With Event Sourcing (ES + CQRS):

Write store = event log (append-only)
Read store = projected views from events
Any read model can be rebuilt by replaying events

Interview tip: "CQRS is overkill for simple CRUD. Recommend it when: read QPS >> write QPS by 100×, read query patterns are complex, or read and write need independent scaling."

Distributed Systems Concepts

Q44. What is consistent hashing?

Pattern:
Consistent
hashing

Difficulty: Hard

Problem: when you add/remove a server, naive hashing ($\text{server} = \text{hash}(\text{key}) \% N$) remaps almost ALL keys → massive cache miss storm.

Consistent Hashing:

Hash servers and keys onto the same ring $[0, 2^{32})$
Key assigned to NEAREST server clockwise on ring
Adding server: only keys between (new server, predecessor) remapped
Removing server: only keys on that server remapped to successor
Impact: only K/N keys remapped (K =keys, N =servers)

Virtual nodes:

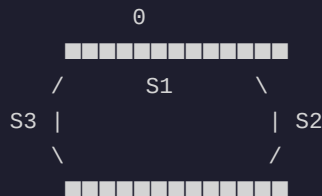
Without VNodes: non-uniform key distribution (servers on ring unevenly)
With VNodes: each server = 150 virtual nodes on ring
→ Keys distributed evenly even with few real servers
→ New server takes ~equal share from all existing servers

Used by:

- Amazon DynamoDB (original Dynamo paper)
- Apache Cassandra (token ring)
- Redis Cluster (hash slots = discretized consistent hashing)
- Nginx upstream hash
- CDN server selection

Consistent hashing with bounded loads:

- Extension: no server handles more than $(1+\epsilon) \times$ average load
- Prevents hot servers when one server is over-assigned



Interview tip: "Draw the ring. Explain VNodes. Know the $O(\log N)$ lookup with sorted array of ring positions. Interviewers love to ask: 'What happens when a node fails?' → successor takes over."

Q45. What is a consensus algorithm and why is Raft important?

Pattern:
Consensus

Difficulty: Hard

Consensus: getting distributed nodes to agree on a value despite failures.

Foundation of: leader election, distributed locks, replicated state machines.

Paxos: original consensus algorithm. Notoriously hard to understand/implement.

Raft: designed for understandability. Same safety guarantees as Paxos.

Raft concepts:

- States: Follower, Candidate, Leader
- Term: monotonically increasing epoch (like a clock)
- Log: ordered sequence of entries (commands to apply)

Leader election:

- Follower → no heartbeat timeout → becomes Candidate
- Candidate → sends RequestVote to all → majority vote → becomes Leader
- Leader → sends heartbeats → prevents new elections

Log replication:

- Leader → AppendEntries to all followers
- Majority ACK → entry committed → applied to state machine
- Quorum = $N/2 + 1$ (for $N=3$: 2 nodes must agree)

Safety: a committed entry will never be lost as long as majority survives

Used by:

- etcd (Kubernetes control plane)
- CockroachDB
- TiKV
- Consul
- RabbitMQ (new quorum queues)

Quorum sizes:

- 3 nodes: tolerate 1 failure

5 nodes: tolerate 2 failures
Rule: tolerate F failures $\rightarrow$ need $2F+1$ nodes

Interview tip: "You don't need to implement Raft, but know: leader election, quorum writes, log replication. 'Why use etcd?' $\rightarrow$ distributed consensus for Kubernetes state. Raft is the foundation."

Q46. What is the two-phase commit (2PC) protocol?

Pattern:
Distributed
transactions

Difficulty: Hard

2PC: protocol for distributed atomic commit across multiple nodes.

Phase 1 (Prepare/Voting):

Coordinator $\rightarrow$ "Prepare?" $\rightarrow$ Participant 1, 2, 3

Each participant: lock resources, write to WAL, reply "Yes" or "No"

Phase 2 (Commit/Rollback):

All voted Yes $\rightarrow$ Coordinator $\rightarrow$ "Commit" $\rightarrow$ release locks

Any voted No $\rightarrow$ Coordinator $\rightarrow$ "Abort" $\rightarrow$ rollback

Problems with 2PC:

Blocking protocol: if coordinator crashes after Prepare but before Commit, participants hold locks indefinitely $\rightarrow$ deadlock

Single point of failure: coordinator is critical path

Latency: 2 round trips $\rightarrow 2 \times$ network RTT overhead

Solution: coordinator writes Commit decision to WAL before sending
 $\rightarrow$ Recovers after restart

3PC (Three-Phase Commit): adds "PreCommit" phase to reduce blocking

Still has issues under network partitions $\rightarrow$ rarely used in practice

Practical alternatives:

Saga pattern: compensating transactions (no locking)

Outbox pattern: atomic DB write + event for eventual consistency

Google Spanner: TrueTime + 2PC with external consistency

When 2PC is appropriate:

Single-vendor, low-latency intra-datacenter (XA transactions in databases)

Not for cross-service microservices communication

Interview tip: "State the 2PC blocking problem immediately. 'Coordinator crashes in Phase 2 $\rightarrow$ participants blocked.' This shows you know the weakness. Then propose Saga as the modern alternative."

Q47. What is the difference between strong and eventual consistency?

Pattern:
Consistency
models

Difficulty:
Medium

Consistency spectrum (strongest to weakest):

Linearizability (strongest):

Every operation appears atomic and instantaneous

All clients see same order of operations

As if only one copy of data

Cost: global coordination $\rightarrow$ high latency

Used by: Zookeeper, etcd, Google Spanner

Sequential Consistency:

All clients see same order of operations
 But operations not necessarily instantaneous
 Used by: some distributed memory systems

Causal Consistency:

Operations that are causally related appear in order
 Concurrent operations may appear in different orders
 Used by: MongoDB (causal sessions), DynamoDB (conditional writes)

Read-Your-Writes:

Client always sees its own writes
 Other clients may see stale data
 Practical minimum for user-facing systems

Eventual Consistency (weakest):

If no new writes, all replicas will eventually converge
 No guarantee on when convergence happens
 Used by: DynamoDB, Cassandra, DNS

Choosing consistency:

Inventory, balances: Linearizable (last-item race condition)
 Social feed: Eventual (user sees your post 200ms later? Fine)
 Shopping cart: Read-Your-Writes (you add item, you see it)
 Friend list: Causal (if A unfriends B, B shouldn't see A's posts)

Interview tip: "Eventual consistency doesn't mean 'broken.' DNS is eventually consistent and works fine. The question is: 'What's the business impact of stale data?' Frame your choice around that."

Q48. What is a distributed lock and when do you need one?

Pattern:
Distributed
locking

Difficulty: Hard

Distributed lock: ensure only one process executes a critical section across multiple machines.

When needed:

- Scheduled job must run on exactly one node
- Inventory: decrement only once despite concurrent requests
- Idempotent processing: exactly-once execution
- Leader election

Redis-based lock (Redlock / SET NX):

SET lock:{resource} {token} NX PX {ttl_ms}
 NX = only set if not exists
 PX = expire in milliseconds
 token = random UUID (identify owner)
 Release: Lua script to delete only if token matches

Problems with Redis single-node lock:

If Redis primary fails before replica syncs → two processes get lock
 Solution: Redlock (acquire lock on $N/2+1$ Redis nodes)

ZooKeeper-based lock (stronger):

Create ephemeral sequential node
 Watch predecessor node
 On predecessor deletion → acquire lock

Fault-tolerant: ephemeral node deleted if holder dies (session expiry)

Lock with lease renewal:

- TTL = 30s, renew every 10s
- Background goroutine sends heartbeat
- If process dies, lock expires automatically

Fencing token:

- Each lock acquisition has increasing ID
- Storage system rejects requests with old fencing token
- Prevents split-brain (two processes think they hold lock)

Interview tip: "Single Redis node lock has failure modes. Mention fencing tokens as the production-safe solution. 'Even if two processes think they have the lock, the fencing token ensures only one actually succeeds at the storage layer.'"

Q49. What is a service mesh?

Pattern:
Service
mesh

Difficulty: Hard

Service mesh: infrastructure layer for service-to-service communication.

Handles: load balancing, service discovery, mTLS, retries, circuit breaking, observability.

Architecture: sidecar proxy (Envoy) injected into every pod.

Application → Sidecar → Network → Sidecar → Application

Application doesn't know about network concerns.

Features:

- Service discovery: sidecar knows about all services via control plane
- Load balancing: L7 (request-level) with health awareness
- mTLS: automatic mutual TLS between all services
- Retries: configurable retry policy per route
- Circuit breaking: auto-open circuit on error threshold
- Traffic splitting: canary deployments (5% to new version)
- Observability: metrics, traces, logs – automatic, without code changes
- Rate limiting: per-service, per-client rate limits

Tools:

- Istio (most popular): complex but full-featured
- Linkerd: lightweight, Go-based, simpler than Istio
- Envoy: the sidecar proxy used by both

When to use a service mesh:

- >10 microservices
- Compliance requires mTLS everywhere
- Need canary deployments
- Cross-language services (mesh is language-agnostic)

When NOT to use:

- Small number of services (overkill)
- Sidecar adds ~2-5ms latency + memory overhead
- Team unfamiliar with service mesh complexity

Interview tip: "Don't bring up service mesh unless asked or for very large systems (50+ services). For an SDE2 interview system design, say 'we'd add Istio as the system grows' rather than designing it upfront."

Q50. What is service discovery?Pattern:
Service
discovery**Difficulty:**
Medium

Service discovery: how services find each other's network addresses.

Challenge: in dynamic systems (Kubernetes), service IPs change constantly.

Client-side discovery:

Client queries service registry → gets list of instances → load balances

Examples: Netflix Eureka + Ribbon (legacy)

Pros: client controls LB algorithm

Cons: registry logic in every client

Server-side discovery:

Client → Load Balancer/API Gateway → registry → routes to instance

Examples: AWS ALB, Kubernetes kube-proxy, Nginx

Pros: client is simple

Cons: extra hop

Service Registry:

Database of service instances (host:port, health status)

etcd, Consul, ZooKeeper, Kubernetes etcd

Kubernetes DNS-based discovery:

Each Service gets DNS: my-service.namespace.svc.cluster.local

DNS resolves to Service ClusterIP → kube-proxy routes to pod

No code changes needed – just call service by name

Self-registration vs Third-party registration:

Self: service registers on startup, deregisters on shutdown

Third-party: orchestrator (Kubernetes) registers on behalf of service

Health checking:

Registry polls /health endpoint every 10s

Unhealthy instances removed from registry

Clients/LBs stop routing to them

Interview tip: "In Kubernetes, service discovery is built-in via DNS. Use `http://payment-service:8080` — Kubernetes handles the rest. Mention this for cloud-native designs."

Q51. What is the difference between synchronous and asynchronous communication?Pattern: Co
mmunicatio
n patterns**Difficulty: Easy**

Synchronous (REST, gRPC):

Caller waits for response

Simple to reason about (request → response)

Tight coupling (caller needs receiver up)

Cascade failures: if B is slow, A is slow

Use: queries needing immediate response, user-facing APIs

Asynchronous (Kafka, SQS, gRPC streaming):

Caller sends and continues (doesn't wait)

Decoupled (receiver can be down, message queued)

Better for: long-running work, fan-out, resilience

Use: notifications, async processing, event sourcing

Sync

Async

Page 35

Free tier: 100 req/min
Pro tier: 10,000 req/min
Enterprise: custom

Interview tip: "Always mention distributed rate limiting requires Redis. Explain the trade-off: Redis-based is slightly slower (+1ms) but correct across all nodes. Local-only can be 10× inaccurate with round-robin LB."

Q53. What is a circuit breaker pattern?

Pattern:
Circuit
breaker

Difficulty:
Medium

Circuit breaker: prevent cascade failures by failing fast.

States:

Closed (normal): requests pass through, failures counted
Threshold: 5 failures in 60s → Open
Open (failing): all requests fail immediately (no service call)
After timeout (30s) → Half-Open
Half-Open (probing): one test request allowed
Success → Closed; Failure → Open again

Benefits:

Fast failure: 1ms rejection vs 30s timeout
Service recovery: allows downstream to recover (no constant hammering)
Cascade prevention: service A failing doesn't take down service B

Metrics to track:

Error rate: failures / total requests in window
Latency p99: slow responses count as failures
Success in half-open: needed to return to Closed

Libraries:

Go: github.com/sony/gobreaker, github.com/afex/hystrix-go
Java: Resilience4j, Hystrix (deprecated)
.NET: Polly

Configuration:

ConsecutiveFailures: 5 (open after 5 consecutive failures)
Interval: 60s (rolling window)
Timeout: 30s (time in Open before trying Half-Open)
MaxRequests: 1 (requests allowed in Half-Open)

Combined with retry:

Retry: transient errors (connection reset, 503) – max 3 times
Circuit breaker: sustained errors → stop retrying entirely
Rule: circuit breaker OUTSIDE retry loop

Interview tip: "State: 'Circuit breaker prevents retry storms. Without it, retrying a broken service creates a DDoS against the recovery.' This shows systems thinking beyond the pattern itself."

Q54. What is a distributed tracing system?

Pattern: Ob
servability

Difficulty:
Medium

Distributed tracing: track a request's journey across multiple services.

Concepts:

Trace: full journey of a request (root span + all child spans)

Span: single operation with timing (DB query, HTTP call, function)
Trace ID: unique ID propagated across all services
Span ID: unique ID for each operation
Parent span ID: links child span to parent

Context propagation (HTTP headers):

X-Trace-ID: 7f3a9b...
X-Span-ID: a3c1f8...
X-Parent-Span-ID: 9b2e4d...
W3C TraceContext standard: traceparent header

Flow:

Client → Service A (creates trace, span 1)
 Service A → DB (span 2, parent=span 1)
 Service A → Service B (span 3, parent=span 1)
 Service B → Redis (span 4, parent=span 3)

OpenTelemetry (OTel): vendor-neutral instrumentation standard

SDK: instrument your code once
Exporter: send to Jaeger, Zipkin, Honeycomb, Datadog, etc.

Go instrumentation:

otelhttp: auto-instrument HTTP servers and clients
otelgrpc: auto-instrument gRPC
otelsql: auto-instrument database/sql

Benefits:

Find slow service in request chain
Debug intermittent errors
Understand service dependencies
Measure p99 latency per service

Tools: Jaeger (open source), Zipkin (open source), Honeycomb, Datadog APM

Interview tip: "Mention OpenTelemetry as the standard — instrument once, send anywhere. The ability to trace a 'slow checkout' across 10 services is what makes distributed tracing invaluable."

Q55. What is the Saga pattern?

Pattern:
Distributed
transactions

Difficulty: Hard

Saga: sequence of local transactions with compensating transactions for failures.
Alternative to 2PC for distributed transactions across microservices.

Example: Order placement

1. Order Service: create order (status=pending)
2. Payment Service: charge card
3. Inventory Service: reserve stock
4. Shipping Service: create shipment

If step 3 fails:

Compensate step 2: refund payment
Compensate step 1: cancel order

Choreography (event-driven):

OrderCreated → PaymentService charges → PaymentCharged
PaymentCharged → InventoryService reserves → StockReserved

StockReserved → ShippingService creates shipment → ShipmentCreated

Failure: StockOutOfStock event →
 PaymentService listens → RefundPayment
 OrderService listens → CancelOrder

Orchestration (central coordinator):
 SagaOrchestrator → calls each service in sequence
 Tracks state of entire saga
 On failure → calls compensating transactions
 Easier to understand, debug, monitor

Challenges:
 Compensating transactions must be idempotent
 Business logic for compensation can be complex
 Partial failures: need clear rollback semantics
 No isolation: interim states visible (order=pending-payment)

Tools:
 Temporal: workflow engine for sagas (Go SDK available)
 AWS Step Functions: managed saga orchestration
 Conductor (Netflix): open source orchestration

Interview tip: "Choreography = loose coupling, harder to trace. Orchestration = central control, easier to debug. Recommend orchestration via Temporal for complex sagas. Mention it by name."

Q56. What is leader election and how does it work?

Pattern:
Leader
election

Difficulty: Hard

Leader election: choose one node from a group to be the primary.
 Used for: scheduled job execution, single-writer constraint, coordinator role.

ZooKeeper (Curator recipe):
 All nodes try to create /election/node_ (ephemeral sequential)
 Node with smallest sequence number = leader
 Others watch the node just before them
 When leader dies, ephemeral node deleted → next watcher becomes leader

etcd (used in Kubernetes):
 Leader acquires lease on a key: PUT /leader/{service} {nodeID} lease=TTL
 Other nodes watch /leader/{service}
 Lease expiry → race to acquire → one wins
 Kubernetes uses this for controller-manager leader election

Redis (Redlock):
 SETNX leader:{service} {nodeID} PX {ttl}
 Renew every TTL/3
 On expiry → next node acquires

Kubernetes leader election (Go):
 client-go/tools/leaderelection
 Uses configmap or lease resource as lock
 Callback: OnStartedLeading / OnStoppedLeading

Failure scenarios:
 Leader dies: lease expires → new leader elected

Time: leader election takes 1-2x lease duration (30-60s typical)

Microservices & APIs

Difficulty: Easy

Interview tip: "Use gRPC for internal microservice communication — it's 2-5× faster and type-safe. Use REST for external APIs. This is the standard production architecture."

Q58. What is GraphQL and when do you use it?Pattern: API
design**Difficulty:**
Medium

GraphQL: query language for APIs. Client specifies exactly what data it needs.

Advantages:

- No over-fetching: get only requested fields
- No under-fetching: single request for nested data (vs multiple REST calls)
- Strongly typed schema (SDL – Schema Definition Language)
- Single endpoint: POST /graphql
- Introspection: clients can query schema itself
- Real-time: subscriptions (WebSocket)

REST vs GraphQL example:

REST to get post with author and comments:

```
GET /posts/42          → {post data}
GET /users/7           → {author data}
GET /posts/42/comments → {comments}
= 3 requests, over-fetching (unused fields)
```

GraphQL:

```
query {
  post(id: 42) {
    title content
    author { name avatar }
    comments(first: 5) { text createdAt }
  }
}
```

= 1 request, exactly what's needed

Challenges:

- N+1 problem: nested resolvers → use DataLoader
- Complex caching: responses vary by query structure
- Authorization: per-field auth complex
- File upload: not native in spec
- Schema evolution: additive only (can't remove fields)

When to use:

- Mobile apps: reduce bandwidth (battery + data)
- Complex/nested data (social networks)
- Rapid frontend iteration (no backend changes for new fields)
- API serving multiple clients with different needs

When NOT to use:

- Simple CRUD API
- File uploads/downloads
- Streaming (use gRPC or WebSocket)

Interview tip: "GraphQL's killer feature is the client specifying exactly what it needs. DataLoader is mandatory for any nested GraphQL resolver — otherwise N+1 queries destroy performance."

Q59. What is an API Gateway pattern?Pattern: API
Gateway**Difficulty:**
Medium

API Gateway: single entry point for all clients.

Handles: routing, auth, rate limiting, SSL, transformation, caching.

Responsibilities:

Routing: /users → UserService, /orders → OrderService
Authentication: verify JWT/API key before forwarding
Rate limiting: per IP/user/API key
SSL termination: HTTPS → HTTP internally
Request/response transformation: REST → gRPC, JSON → Protobuf
Load balancing: distribute across service instances
Caching: cache GET responses
Monitoring: centralized metrics and logging
Circuit breaking: fail fast if downstream unhealthy

BFF (Backend for Frontend):

Separate API Gateway per client type
Mobile BFF: smaller payloads, fewer fields
Web BFF: richer data, more features
Avoids compromise in a single gateway

Tools:

Open source: Kong, Traefik, Nginx
Cloud managed: AWS API Gateway, Azure APIM, GCP Apigee
Service mesh: Istio (handles service-to-service)

Anti-patterns:

Too much business logic in gateway (it should route, not compute)
Single gateway as SPOF (run multiple instances behind LB)
Synchronous gateway for every microservice call (consider async for non-critical)

Design:

Client → [API Gateway: auth + rate limit + route]
→ [UserService] [OrderService] [PaymentService]

Interview tip: "API Gateway is a must in any microservices design. It prevents $N \times M$ connections (N clients $\times$ M services). Without it: every service must implement auth, rate limiting, SSL — code duplication."

Q60. What are the microservices decomposition patterns?

Pattern: Microservices design

Difficulty: Medium

Decompose by business capability:

Domain-aligned services: UserService, OrderService, PaymentService
Each service owns its data (bounded context)
Communication: events or APIs
Best for: stable, well-understood domain

Decompose by subdomain (DDD):

Core domain: competitive advantage (pricing engine, recommendation)
Supporting: generic but important (auth, notifications)
Generic: commodity (email, payments) → use third-party

Strangler Fig pattern (migration):

Legacy monolith → incrementally replace with microservices
New service deployed alongside → traffic gradually shifted
Old monolith functionality removed piece by piece
No big-bang rewrite → risk-free migration

Service granularity guidelines:

Too fine-grained:

- Network overhead for every interaction
- More deployment complexity
- Distributed transactions required

Too coarse-grained:

- Low cohesion, mixed concerns
- Single service can't be scaled independently

Right size: team can own it, deploys independently, single business function

Database patterns:

- Database per service (ideal): complete isolation
- Shared database (anti-pattern): tight coupling
- Shared schema (compromise): different tables, same DB

Inter-service communication:

- Sync: gRPC (low latency, immediate response needed)
- Async: Kafka (decouple, resilience, fan-out)

Interview tip: "The two-pizza rule: team that owns a service should be feedable with two pizzas. If the service needs 20 engineers, it's too big. If it needs 0.1 engineers, it's too small."

Q61. What is the difference between API versioning strategies?

Pattern: API
versioning

Difficulty: Easy

URL path versioning: /api/v1/users, /api/v2/users

- Pros: explicit, easy to route, easy to test
- Cons: URL changes, clients must update

Most common approach

Query parameter: /users?version=2

- Pros: URL stable, version optional
- Cons: less explicit, can be forgotten

Header versioning: Accept: application/vnd.myapi.v2+json

- Pros: URL stable, RESTful
- Cons: less visible, harder to test with browser

Content negotiation: Accept: application/vnd.company.resource-v2+json

- Pros: full REST compliance
- Cons: verbose, complex

Semantic versioning for APIs:

- Major (v1 → v2): breaking changes (remove fields, change types)
- Minor (v1.1 → v1.2): additive changes (new optional fields)
- Patch: bug fixes, no contract change

Breaking vs non-breaking changes:

- Breaking: remove field, change field type, rename endpoint
- Non-breaking: add optional field, add new endpoint, add optional header

API deprecation:

- Sunset header: Sunset: Wed, 11 Nov 2025 23:59:59 GMT
- Deprecation-Notice: deprecated API, migrate to v2
- Minimum deprecation period: 6-12 months for external APIs

Interview tip: "URL path versioning (/v1/, /v2/) is the de-facto standard. GraphQL avoids versioning by making all changes additive. Mention that your API Gateway can route v1 → old service, v2 → new service."

Q62. What is idempotency in APIs?

Pattern: Idempotency

Difficulty: Medium

Idempotent operation: calling it multiple times = same result as calling once.

HTTP methods:

GET: naturally idempotent (just a read)
PUT: replace resource – idempotent
DELETE: delete resource – idempotent (second call: 404 or 200, same state)
POST: NOT idempotent (creates new resource each time)
PATCH: depends on implementation

Making POST idempotent (Idempotency-Key):

Client generates UUID per operation
Sends: POST /payments + Idempotency-Key: uuid-123
Server: check if uuid-123 already processed
First time: process payment, store {uuid-123: response}
Subsequent: return stored response immediately

Database implementation:

```
CREATE TABLE idempotency_keys (  
  key VARCHAR(255) PRIMARY KEY,  
  response JSON,  
  expires_at TIMESTAMP  
);  
  
INSERT INTO idempotency_keys (key, response, expires_at)  
VALUES (?, ?, NOW() + INTERVAL '24 hours')  
ON CONFLICT (key) DO NOTHING  
RETURNING key;
```

If RETURNING returns row → first call → process
If RETURNING returns nothing → duplicate → return stored response

Stripe uses Idempotency-Key header for all payment APIs.

Used by: Stripe, Twilio, Braintree.

Interview tip: "Idempotency-Key is mandatory for payment APIs. Network timeouts mean the client retries — without idempotency, you charge the card twice. Stripe's implementation is the gold standard."

Q63. What is the strangler fig pattern?

Pattern: Migration

Difficulty: Medium

Strangler Fig: incrementally replace a legacy system without big-bang rewrite.

Named after the fig tree that grows around and replaces a host tree.

Steps:

1. Deploy new service alongside monolith
2. Route SOME traffic to new service (start with non-critical features)
3. Gradually increase traffic to new service
4. Remove corresponding code from monolith
5. Repeat until monolith is gone

Implementation:

API Gateway / Nginx as router:
/api/users/profile → new UserService (v2)
/api/users/legacy → old monolith (v1)

Feature flags:

```
if featureFlag("new_user_service") { callNewService() }
else { callMonolith() }
```

Strangler Facade: proxy in front of both

Routes based on feature, URL, or user segment

Traffic migration:

Start: 1% to new service (canary)
Monitor: errors, latency, correctness
Increase: 10% → 50% → 100%
Rollback: flip feature flag if issues

Database migration:

Dual-write: write to both old and new DB
Verify: compare outputs
Cut-over: read from new DB
Stop writing to old DB

Challenges:

Two systems running in parallel (operational overhead)
Data consistency during transition
Performance regression can be hard to spot

Interview tip: "For any 'migrate from monolith' question, the answer is Strangler Fig. Never big-bang rewrite — too risky. Gradual migration lets you validate at each step."

Q64. What is a health check endpoint and why is it important?

Pattern:
Operational

Difficulty: Easy

Health check: endpoint indicating whether a service is healthy.

Types:**Liveness probe (/live):**

Is the process alive? (not deadlocked)
Returns 200: process is alive
Returns 500: process should be restarted
Simple: if we can respond, we're alive

Readiness probe (/ready):

Can we serve traffic? (DB connected, cache warmed)
Returns 200: ready for traffic
Returns 503: not ready — remove from LB rotation
Checks: DB connection, Redis connection, required dependencies

Startup probe:

Has initial startup completed?
Delays liveness/readiness until startup done
Prevents killing slow-starting services

Implementation in Go:

```
func (s *Server) livenessHandler(w http.ResponseWriter, r *http.Request) {
    w.WriteHeader(http.StatusOK)
    json.NewEncoder(w).Encode(map[string]string{"status": "ok"})
}

func (s *Server) readinessHandler(w http.ResponseWriter, r *http.Request) {
```

```

    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    if err := s.db.PingContext(ctx); err != nil {
        w.WriteHeader(http.StatusServiceUnavailable)
        json.NewEncoder(w).Encode(map[string]string{"status": "db_unavailable"})
        return
    }
    w.WriteHeader(http.StatusOK)
}

```

Kubernetes:

livenessProbe: restarts container if failing
 readinessProbe: removes pod from Service endpoints if failing
 Both check /live and /ready respectively

Interview tip: "Liveness vs readiness is a common Kubernetes interview question. Liveness = 'should this pod be restarted?' Readiness = 'should this pod receive traffic?' Don't conflate them."

Q65. What is canary deployment?

Pattern:
Deployment
strategy

Difficulty:
Medium

Canary deployment: deploy new version to small subset of users first.
 Named after canary-in-coal-mine safety test.

Strategies:

Percentage-based: 1% → 5% → 25% → 100%
 User segment: internal users → beta users → all users
 Geographic: deploy to us-west-2 first, then other regions
 Random: $\text{hash}(\text{user_id}) \% 100 < 5 \rightarrow \text{new version}$

Implementation:

Kubernetes + Argo Rollouts:

```

progressDeadlineSeconds: 600
steps:
- setWeight: 5      # 5% to canary
- pause: {duration: 10m}
- setWeight: 20     # 20% to canary
- pause: {duration: 10m}
- setWeight: 100    # full rollout

```

Nginx/Envoy traffic splitting:

```

upstream old { server old-service:8080 weight=95; }
upstream new { server new-service:8080 weight=5; }

```

Feature flags (LaunchDarkly, Unleash):

```

if featureFlag.Enabled("new-checkout", userID) { → new }
else { → old }

```

Monitoring during canary:

Error rate: canary error rate must be $\leq$ production
 Latency: p99 canary $\leq$ production p99
 Business metrics: conversion rate, revenue per session
 Automatic rollback if thresholds breached

Blue-Green deployment (alternative):

Two identical environments: Blue (current) and Green (new)

Switch LB from Blue to Green instantly
Instant rollback: switch back to Blue
Cost: 2× infrastructure

Interview tip: "Canary deployment is how Netflix/Uber/Amazon ship code safely. Combined with automatic rollback on error rate increase, it catches bugs before they affect all users."

Observability & Reliability

Q66. What are the three pillars of observability?

Pattern: Observability

Difficulty: Easy

Observability: ability to understand internal state from external outputs.

Three pillars:

1. Metrics (quantitative, aggregated):
 - What: numeric measurements over time
 - Examples: QPS, error_rate, p99_latency, memory_usage, cache_hit_rate
 - Storage: Prometheus (pull-based), StatsD (push-based), CloudWatch
 - Visualization: Grafana dashboards
 - Alerting: alert when error_rate > 1% for 5min
2. Logs (events, text):
 - What: timestamped records of events
 - Examples: "user 42 logged in", "payment failed: insufficient funds"
 - Structured logging: JSON (queryable by field)
 - Storage: Elasticsearch, Loki, CloudWatch Logs, Splunk
 - Best practice: correlation ID / trace ID in every log line
3. Traces (distributed, causal):
 - What: end-to-end request journey across services
 - Examples: checkout → payment (50ms) → inventory (10ms) → shipping (20ms)
 - Storage: Jaeger, Zipkin, Honeycomb, Datadog APM
 - Standard: OpenTelemetry (instrument once, export anywhere)

The three together:

Metrics tell you SOMETHING is wrong (error rate up)
Logs tell you WHAT went wrong (specific error messages)
Traces tell you WHERE it went wrong (which service/span)

RED method (services):

Rate: requests per second
Errors: error rate
Duration: latency distribution

USE method (resources: CPU, disk, network):

Utilization: % resource used
Saturation: queue depth, waiting
Errors: error rate

Interview tip: "Always mention all three pillars in any reliability discussion. Add: 'We'd use OpenTelemetry for unified instrumentation — instrument once, send to any backend.'"

Q67. What is SLI, SLO, and SLA?Pattern:
SRE
concepts**Difficulty: Easy**

SLI (Service Level Indicator): measurable metric of service behavior
Examples: availability, latency, error rate, throughput

SLO (Service Level Objective): target value for SLI
Examples: "99.9% availability", "p99 latency < 200ms", "error rate < 0.1%"
Internal goal – NOT a customer commitment (that's SLA)

SLA (Service Level Agreement): contractual commitment with consequences
If SLA breached: financial penalties, service credits
Example: "99.9% availability or 10% service credit"
 $SLA \leq SLO$ (SLO is stricter – gives buffer before SLA breach)

Error budget:
Error budget = 1 - SLO
99.9% SLO → 0.1% error budget = 8.77 hours/year
Error budget consumed: actual downtime / total budget
If error budget consumed → freeze new deployments
If error budget healthy → ship faster, take more risk

Practical SLOs:
Availability: 99.9% (3 nines) for most services
Latency p99: 200ms for APIs, 1s for complex queries
Error rate: < 0.5%
Freshness: data updated within 5 minutes

Multi-window alerting:
Alert fast: 2% error rate in last 5 minutes (fast burn)
Alert slow: 0.5% error rate in last 6 hours (slow burn)
Both needed: fast catches spikes, slow catches slow leaks

Interview tip: "Error budget is the key concept interviewers test. 'How do you balance feature velocity with reliability?' → Error budget: when it's healthy, ship features; when it's exhausted, fix reliability."

Q68. What is the difference between horizontal and vertical partitioning?Pattern:
Data
partitioning**Difficulty:
Medium**

Horizontal partitioning (Sharding):
Split rows across multiple tables/databases
Each shard has same schema, different data
Example: users 1-1M in shard1, 1M-2M in shard2
Reduces: rows per table, query time, storage per node
Increases: write throughput, can store more data

Vertical partitioning:
Split columns across multiple tables
Related/frequently-accessed columns together
Example:
users_core: (id, name, email, created_at) – queried frequently
users_profile: (id, bio, avatar, settings) – queried less often
users_metrics: (id, login_count, last_ip) – updated frequently
Reduces: table width, I/O for narrow queries
Useful for: wide tables, column access patterns differ

Functional partitioning:

Split by feature/domain into separate databases
 user_db: all user data
 order_db: all order data
 payment_db: all payment data
 = microservices DB-per-service pattern

Partition pruning (query optimisation):

PostgreSQL table partitioning:
 PARTITION BY RANGE (created_at)
 Partition for each month
 Query for last month only scans last month's partition
 Huge savings for time-series data

Partition key selection:

Access pattern drives key selection
 If 90% of queries filter by user_id → shard/partition by user_id

Interview tip: "Horizontal = row-based sharding. Vertical = column splitting. Know both. PostgreSQL declarative partitioning (RANGE/LIST/HASH) is the production implementation of horizontal partitioning."

Q69. What is chaos engineering?

Pattern:
Reliability
engineering

Difficulty:
Medium

Chaos engineering: intentionally inject failures to find weaknesses before they cause outages.

Principles (Netflix Chaos Monkey paper):

Build hypothesis: "System maintains 99.9% availability when one AZ fails"
 Vary real-world events: server crash, network partition, high latency
 Run in production (or staging): real behavior, real dependencies
 Minimize blast radius: start small, monitor closely
 Automate to run continuously

Types of failure injection:

Resource failure: kill random pods/VMs (Chaos Monkey)
 Network: add latency, packet loss, partition between services
 CPU/Memory: spike CPU, fill memory
 Clock skew: advance system clock
 Dependency failure: make Redis unresponsive, DB connection refused
 AZ failure: shut down all instances in one availability zone

Tools:

Chaos Monkey (Netflix): kills EC2 instances randomly
 Gremlin: comprehensive chaos platform
 Chaos Toolkit: open source, multiple plugins
 Litmus (Kubernetes): CNCF chaos engineering tool
 Istio fault injection: inject HTTP errors/delays at mesh level

Controlled experiment:

Define: "payment service handles Redis failure gracefully"
 Baseline: measure normal p99 latency, error rate
 Inject: make Redis unresponsive for 60 seconds
 Observe: does circuit breaker open? does it fall back to DB?
 Conclusion: pass/fail against hypothesis

Game day: scheduled exercise where team intentionally causes failures

Interview tip: "Chaos engineering is practiced by Netflix, AWS, Google. For SDE2: know what it is and why. 'We use Chaos Monkey to ensure our circuit breakers and fallbacks actually work in production.'"

Q70. What is on-call and incident management?

Pattern:
Operational
excellence

Difficulty: Easy

On-call: engineers are reachable 24/7 to respond to production incidents.

Severity levels:

- SEV1 (Critical): complete outage, data loss, security breach
 - Response: immediate, all hands
 - Notify: CTO, customers, status page
- SEV2 (Major): major feature broken, significant user impact
 - Response: < 15 minutes
 - Notify: engineering manager
- SEV3 (Minor): degraded performance, partial feature unavailable
 - Response: business hours
- SEV4 (Cosmetic): UI issue, minor bug
 - Schedule: next sprint

Incident response steps:

1. Acknowledge: claim the incident, prevent duplicate response
2. Assess: understand impact (how many users? revenue?)
3. Communicate: update status page, notify stakeholders
4. Investigate: use metrics + traces + logs to find root cause
5. Mitigate: fastest fix (rollback, feature flag off, scale up)
6. Resolve: permanent fix
7. Post-mortem: blameless analysis within 48 hours

Runbook: documented step-by-step response procedure for known incidents

"Redis is full → check largest keys → flush expired → page DBA"

Post-mortem (blameless):

- Timeline of events
- Root cause analysis (5 Whys)
- Contributing factors
- Action items (prevent recurrence)
- No blame, no finger-pointing – systems, not people

Tools: PagerDuty, OpsGenie (alerting), Statuspage (communication)

Interview tip: "Blameless post-mortem culture prevents cover-up and enables learning. 'We don't fire people for outages — we fix the system.' This is the SRE philosophy."

Q71. What is blue-green deployment?

Pattern:
Deployment

Difficulty: Easy

Blue-Green: two identical production environments.

Blue: currently live (100% traffic)

Green: new version deployed (0% traffic)

Process:

1. Deploy new version to Green (while Blue serves all traffic)
2. Run smoke tests against Green
3. Flip LB: route 100% traffic to Green
4. Blue kept warm for rollback

5. After validation: decommission Blue / make it next staging

Rollback: flip traffic back to Blue (seconds, not hours)

vs Canary:

Blue-Green: instant switch, all-or-nothing

Canary: gradual traffic shift, catches issues earlier

Challenges:

Database migrations: schema changes must be backward compatible

Blue and Green may run simultaneously during switch

Solution: expand-contract migrations (multi-step schema changes)

2× infrastructure cost during deployment

Session state: active sessions on Blue lost on switch

Solution: use Redis for sessions (shared, not per-server)

Kubernetes:

Two Deployments (blue and green)

Service selector switches between them

```
kubectl patch service myapp -p '{"spec":{"selector":{"version":"green"}}}'
```

Database schema migration for Blue-Green:

Step 1: Add new column (backward compatible)

Step 2: Deploy new version using new column

Step 3: Backfill data

Step 4: Remove old column (after old version retired)

= Expand-Contract pattern

Interview tip: "Blue-Green requires backward-compatible DB schemas during the window when both versions run. Expand-contract (Parallel Change) pattern solves this. Know it."

Q72. How do you design for high availability?

Pattern:
High
availability

Difficulty:
Medium

High Availability: system continues operating despite failures.

Target: 99.99% (52 min downtime/year) or better.

Key principles:

Eliminate single points of failure (SPOF)

Redundancy at every tier

Health checks + automatic failover

Multi-AZ / multi-region deployment

Stateless services (any instance can handle any request)

Graceful degradation (partial functionality, not complete outage)

Layer by layer:

DNS: multi-AZ Route53 with health checks

Load Balancer: multiple LB instances (AWS ALB is managed/auto-HA)

Application: multiple instances across AZs, auto-scaling

Cache: Redis Sentinel or Cluster (automatic failover)

Database: primary + replica in different AZs, automated failover (RDS Multi-AZ)

Storage: S3 (11 nines durability), replicated across AZs

Active-Active vs Active-Passive:

Active-Active: all nodes serve traffic simultaneously

Better utilisation, immediate failover

Requires conflict resolution for writes

Active-Passive: one node serves traffic, other on standby

Simpler, no conflict resolution

Failover time = detection + switchover (~60s)

RTO and RPO:

RTO (Recovery Time Objective): maximum acceptable downtime

RPO (Recovery Point Objective): maximum acceptable data loss

"RTO=1h, RPO=1min" means: back up within 1 hour, lose max 1 min of data

Multi-region:

Active-Active: Route53 Latency routing, both regions serve traffic

Active-Passive: Route53 Failover routing, secondary on standby

Interview tip: "State RPO and RTO for your design. 'This design has RTO=30s (Aurora failover time) and RPO=0 (synchronous replication).' Numbers make your answer concrete."

Q73. What is graceful degradation?

Pattern:
Resilience

Difficulty:
Medium

Graceful degradation: when components fail, serve degraded but functional experience.
vs Hard failure: complete service outage.

Examples:

Netflix: recommendations service down → show generic popular titles, not error page

Twitter: timeline aggregation slow → show cached timeline, not spinner

E-commerce: inventory service down → show product, remove "in stock" indicator

Search: ML ranking service down → fall back to simple chronological results

Implementation patterns:

Fallback cache: serve stale data from cache if primary source fails

Default value: if personalisation fails, serve default content

Feature flag disable: turn off non-essential features under load

Queue and retry: accept request, process later when service recovers

Static fallback: serve pre-rendered static page if dynamic fails

Circuit breaker for graceful degradation:

```
func getRecommendations(userID string) []Product {
    result, err := circuitBreaker.Execute(func() ([]Product, error) {
        return recommendationService.Get(userID)
    })
    if err != nil {
        return popularItems.Get() // fallback: popular items
    }
    return result
}
```

Timeout hierarchy:

User-facing API timeout: 1s

Downstream service timeout: 500ms (gives time for fallback)

DB query timeout: 200ms

SL0 for degraded mode:

Normal: 99.9% of requests < 100ms

Degraded (with fallback): 99.9% of requests < 200ms (but no outage)

Interview tip: "Graceful degradation shows operational maturity. For any design, ask: 'What happens when component X fails? What does the user see?' Good systems degrade gracefully, not catastrophically."

Classic System Designs

Q74. Design a URL shortener (like bit.ly).

Pattern:
Classic
design

Difficulty:
Medium

Requirements:

Functional: shorten long URL, redirect to original, analytics

Non-functional: 100M URLs, low latency redirect (<10ms), high availability

Estimation:

Writes: 1M new URLs/day = 11 QPS

Reads: 100M redirects/day = 1,160 QPS (100:1 read:write)

Storage: $1M \times 365 \times 5yr \times 500B = \sim 900$ GB

Short code generation:

Option 1: MD5(longURL) → take first 7 chars (collision risk)

Option 2: Counter-based → base62(auto-increment_id)

1 = "1", 62 = "Z", 3.5B = 7 chars in base62

Option 3: Pre-generate random 7-char codes, store in unused pool

API design:

POST /shorten {longURL} → {shortURL: "https://bit.ly/Ab3cX9Y"}

GET /{code} → 301/302 redirect to longURL

Data model:

urls table: code VARCHAR(7) PK, long_url TEXT, user_id, created_at, expires_at, clicks

Architecture:

Client → LB → API Servers → [Cache: Redis] → DB (PostgreSQL/DynamoDB)

Redirect path: check Redis first (sub-ms) → miss → DB → cache for next time

Cache strategy:

Cache hot URLs (top 20% = 80% of traffic)

Key: short_code, Value: long_url, TTL: 24h

Cache hit rate: ~80%

Redirect: 301 (permanent, cached by browser) vs 302 (temporary, analytics see every visit)

For analytics: 302

Analytics:

Async: log click event to Kafka → consumer updates click_count

Don't synchronously update DB on every redirect (bottleneck)

Custom short codes: allow user to specify → check uniqueness

Expiration: soft delete via expires_at column

Interview tip: "Interviewers love to ask: '301 vs 302?' Answer: 301 is cached by browser (good for performance, bad for analytics). 302 hits your server every time (good for analytics). Bit.ly uses 302 for analytics."

Q75. Design a rate limiter.Pattern:
Classic
design**Difficulty:**
Medium

Requirements:

Functional: limit requests per IP, per user, per API key

Non-functional: low latency (<5ms overhead), distributed (multiple servers)

Rate limiting algorithms:

Token bucket: allow burst, avg rate enforced → most flexible

Sliding window counter: approximate, low memory → good balance

Sliding window log: exact, high memory → use for payments only

Architecture:

[Client] → [LB] → [API Gateway with rate limiter] → [Services]

Rate limiter checks: `Redis.INCR(key, window) < limit`

Redis sliding window counter:

Key: `"ratelimit:{user_id}:{window_start}"`

INCR key → get count

EXPIRE key TTL

If count > limit → 429

Rule storage:

Config service or DB: `{endpoint: "/checkout", limit: 10, window: "1min"}`

Cached in API Gateway memory, refreshed every 60s

Distributed consideration:

Each API server has local counter (inaccurate with N servers)

Use Redis: single source of truth → accurate but +1ms latency

Lua script for atomic INCR + check:

`local count = redis.call('INCR', key)``if count == 1 then redis.call('EXPIRE', key, ttl) end``return count`

Response headers:

`X-RateLimit-Limit: 100``X-RateLimit-Remaining: 45``X-RateLimit-Reset: 1609459200``Retry-After: 60 (on 429)`

Scale:

Redis cluster for rate limit keys

Shard by `{user_id}` hash for even distribution

Read replicas for high-QPS rate check (but requires async counter → less accurate)

Interview tip: "The Lua script for atomic INCR + EXPIRE is the production implementation. Without atomicity: two goroutines both check limit 99 → both pass → 101 requests allowed. Lua prevents this race."

Q76. Design a key-value store (like Redis).Pattern:
Classic
design**Difficulty: Hard**

Requirements:

Get(key), Set(key, value), Delete(key), TTL support

Non-functional: <1ms p99, 1M ops/sec, durable, replication

Single-node architecture:

In-memory hash table: $O(1)$ get/set
 WAL (Write-Ahead Log): durability
 Background snapshot (RDB): periodic full dump
 LRU eviction when memory full

Data structures:

String: raw bytes (key $\rightarrow$ byte[])
 Hash: key $\rightarrow$ hash map
 List: key $\rightarrow$ doubly-linked list
 Sorted Set: key $\rightarrow$ skip list + hash map
 (see Redis internals)

Persistence:

AOF (Append-Only File): log every command $\rightarrow$ replay on restart
 Durable: lose at most last fsync window (1s by default)
 RDB (snapshot): periodic dump $\rightarrow$ fast startup, may lose minutes of data
 Both: RDB for fast startup + AOF for durability (Redis default in production)

Replication:

Primary handles writes + reads (or separate replicas for reads)
 Replicas receive WAL stream from primary
 Eventual consistency: replication lag possible
 Sentinel: automatic failover on primary death

Cluster (horizontal scale):

16,384 hash slots across N nodes
 Each node owns a range of slots
 Client-side routing: client gets MOVED redirect if wrong node

Memory optimization:

Compressed encoding for small objects (ziplist, intset)
 Shared integers (0-9999 preallocated)
 LZF compression for large strings

Interview tip: "Don't reinvent Redis in the interview — explain its design. Focus on: in-memory hash table, AOF+RDB persistence, primary-replica replication, Sentinel for HA. These are the key design decisions."

Q77. Design a notification system.

Pattern:
Classic
design

Difficulty:
Medium

Requirements:

Send push, email, SMS, in-app notifications
 Support: immediate, scheduled, user preferences
 Non-functional: 1M notifications/day, low latency, high deliverability

Estimation:

1M notif/day = ~12 QPS average, 100 QPS peak
 Fanout example: 100K users following celebrity $\rightarrow$ 100K push notifications

Architecture:

Client $\rightarrow$ API $\rightarrow$ Notification Service $\rightarrow$ Kafka
 $\rightarrow$ Email worker $\rightarrow$ SendGrid/SES
 $\rightarrow$ Push worker $\rightarrow$ FCM/APNs
 $\rightarrow$ SMS worker $\rightarrow$ Twilio

Data model:

```
notifications: (id, user_id, type, template_id, payload, scheduled_at, sent_at, status)
user_preferences: (user_id, channel, enabled, quiet_hours_start, quiet_hours_end)
devices: (user_id, device_token, platform, created_at)
```

User preferences check:

```
Is notification enabled for this channel?
Is user in quiet hours? (if low priority, defer)
Rate limiting: max 5 push per hour
```

Template system:

```
Templates stored in DB: "Hello {{name}}, your order {{order_id}} shipped!"
Render at send time with actual data
Supports i18n (language per user preference)
```

Delivery guarantees:

```
Kafka for durability: message not lost even if worker crashes
Retry: failed delivery retried 3× with backoff
DLQ: after 3 failures → dead letter queue → alert
```

Tracking:

```
Delivered, Opened, Clicked events
Webhook from FCM/APNs on delivery/failure
Store in ClickHouse for analytics (not PostgreSQL — high volume)
```

Priority queues:

```
High priority (OTP, password reset): separate fast queue
Low priority (marketing): bulk queue, send during off-peak
```

Interview tip: "Template + user preferences + delivery guarantees = full notification system. Highlight the priority queue for OTPs — users won't wait 5 minutes for a one-time password."

Q78. Design Twitter's timeline.

Pattern:
Classic
design

Difficulty: Hard

Requirements:

```
Post tweet, follow/unfollow, view home timeline (reverse chron)
Non-functional: 300M DAU, 500M tweets/day, <500ms timeline load
```

Timeline approaches:

Fan-out on write (push model):

```
When Alice tweets → precompute timeline for all followers
Store tweet_id in each follower's timeline cache (Redis list)
On read: fetch list of tweet_ids → batch fetch tweets → render
Pros: read is fast (pre-computed)
Cons: celebrities with 100M followers → 100M writes per tweet
```

Fan-out on read (pull model):

```
When user opens timeline → fetch all followees' tweets → merge sort
Pros: no write amplification
Cons: slow for users with many followees (thousands of followees)
```

Twitter's hybrid approach:

```
Normal users: fan-out on write (most users have <10K followers)
Celebrities (>1M followers): fan-out on read
On timeline read: merge pre-computed timeline (cache) + celebrity tweets
```

Data model:

tweets: (tweet_id, user_id, content, created_at) [tweet snowflake ID encodes time]
 followers: (follower_id, followed_id)
 timeline_cache: Redis sorted set per user (tweet_id → timestamp)

Timeline read:

ZREVRANGE timeline:{user_id} 0 19 → 20 tweet IDs
 Batch fetch from tweets table/cache
 Inject celebrity tweets → merge by timestamp
 Return

Write path:

POST /tweet → write to tweets table → enqueue fan-out job
 Fan-out worker: get followers → for each → ZADD timeline:{follower_id} timestamp tweet_id

Cache:

timeline_cache in Redis: last 1000 tweets per user
 tweet details in Memcached (tweet_id → tweet object)

Scale:

Tweets: write to Cassandra (time-series, write-heavy)
 Timeline: Redis cluster (sorted sets per user)
 Search: Elasticsearch

Interview tip: "The fan-out hybrid model (write for regular, read for celebrities) is the key insight Twitter uses. Know it verbatim — it's asked constantly."

Q79. Design a distributed file storage system (like S3).

Pattern:
 Classic
 design

Difficulty: Hard

Requirements:

Upload/download files, list files, delete
 Non-functional: 1 billion files, 100 TB, 99.999999999% (11 nines) durability

Architecture:

Client → LB → API Gateway
 API Gateway → Metadata Service (which chunk → which server)
 → Chunk Servers (actual data storage)

Chunking:

Split file into 64MB chunks
 Each chunk replicated 3× across 3 chunk servers
 Checksum per chunk (MD5/SHA256) for integrity verification

Metadata Service (Zookeeper + DB):

file_id → {file_name, size, owner, chunks: [chunk_id, ...]}
 chunk_id → {servers: [server1, server2, server3], checksum}

Chunk server:

Stores chunks as local files on disk
 Heartbeat to metadata service every 10s
 GC: remove orphan chunks (no reference in metadata)

Upload flow:

Client → API → Metadata: allocate chunk IDs + servers
 Client → Chunk servers: upload chunks in parallel

Metadata: mark file complete

Download flow:

Client → Metadata: get chunk_id → server mappings
Client → Chunk servers: download chunks in parallel
Client: reassemble + verify checksums

Replication strategy:

Cross-AZ: chunks on servers in different AZs
Rack-aware: different racks within same AZ
Repair: background scanner verifies checksums → re-replicates missing chunks

S3 uses Reed-Solomon erasure coding:

vs 3× replication: 6-of-9 coding uses 1.5× storage vs 3× storage
Rebuild from any 6 of 9 shards

Interview tip: "Durability = checksums + replication + background integrity scanning. '11 nines durability' = if you stored 10 billion files for 10,000 years, you'd expect to lose one. Reed-Solomon is how S3 achieves it."

Q80. Design a search engine (like Elasticsearch/Google Search).

Pattern:
Classic
design

Difficulty: Hard

Requirements:

Index documents, full-text search, relevance ranking
Non-functional: 1B documents, sub-100ms query, near-real-time indexing

Core components:

1. Document Store:

Stores raw documents: {doc_id, url, title, body, metadata}
DynamoDB or Cassandra (key = doc_id, read by doc_id)

2. Inverted Index (heart of search engine):

term → posting list [{doc_id, TF, positions}, ...]
TF = term frequency in document
Built from tokenized, normalized text (lowercase, stem, stop words removed)
Stored on disk in immutable segments (SSTable-like)

3. Indexing Pipeline:

Document → Tokenizer → Stemmer → Stop word filter → Inverted index update
Batch indexing: MapReduce over all documents
Incremental: new documents → Kafka → indexing service

4. Query Processing:

Query tokenized → look up each term → intersect posting lists (AND)
Scoring: BM25 (modern TF-IDF variant)
score = $IDF \times TF \times \text{field length normalisation}$
Rank by score → return top K

5. Distributed Search:

Sharding: documents distributed across N shards (by doc_id hash)
Query: broadcast to all shards → each returns top K → merge sort → global top K
Replication: each shard replicated 3×

6. Caching:

Popular queries cached in Redis (query → results)

TTL: 5 minutes (results may change)

Elasticsearch architecture:

Index → Shards → Segments (Lucene)

Each shard = Lucene index (inverted index + document store)

Refresh interval: 1s (near real-time)

Interview tip: "Inverted index is the core data structure. Know: term → posting list. Know BM25 scoring. Know sharded search (scatter-gather pattern). These three cover 80% of search engine questions."

Q81. Design a ride-sharing system (like Uber).

Pattern:
Classic
design

Difficulty: Hard

Requirements:

Rider requests ride, match with nearby driver, real-time tracking

Non-functional: 1M concurrent rides, <2s matching, GPS update every 5s

Estimation:

1M active drivers, each updates location every 5s

Location write QPS: $1M / 5 = 200,000$ writes/sec

Architecture:

Rider App → LB → [API Gateway] →

Location Service (GPS updates from drivers)

Matching Service (find nearby driver)

Trip Service (manage active trips)

Payment Service (charge on trip complete)

Notification (push to rider/driver)

Location Service:

Driver app → POST /location {lat, lng, timestamp}

Store in Redis geospatial (GEOADD driver:available lat lng driver_id)

GEOSEARCH: find drivers within 2km of rider's location

Redis GEODIST, GEOSEARCH: $O(N + \log(M))$ where M =elements in radius

Geospatial indexing:

Redis GEOSEARCH (recommended):

GEOSEARCH drivers:available FROMLONLAT lng lat BYRADIUS 2 km

Google S2 Library (Uber's approach):

Earth divided into cells at different levels

Level 12 cell $\approx 2\text{km}^2$ → hash driver into cells

Find drivers: look up rider's cell + neighboring cells

Matching algorithm:

Get available drivers in 2km radius

Score: distance + driver rating + ETA

Assign best driver (lock in DB to prevent double-assignment)

Race condition: two riders match same driver → use DB transaction/lock

Trip management:

Trip states: requested → accepted → arriving → in_progress → completed

Each state change → event to Kafka → notification service

Real-time tracking:

WebSocket connection between driver app and rider app (via server)

Driver → server → Kafka → push to rider's WebSocket

Map updates every 5s with driver's GPS

Surge pricing:

demand = ride requests in area / supply = available drivers
surge_multiplier = f(demand / supply)
Recalculate every minute per geo-cell

Interview tip: "Geospatial indexing is the core challenge. Redis GEOSEARCH or S2 library are the two main approaches. Uber uses H3 (hexagonal hierarchy) for their zone system. Know at least one approach deeply."

Q82. Design a video streaming platform (like YouTube).

Pattern:
Classic
design

Difficulty: Hard

Requirements:

Upload video, stream video, search, recommendations
Non-functional: 500M DAU, 500 hours of video uploaded/minute, 1B hours watched/day

Upload pipeline:

1. User uploads raw video → Object store (S3)
2. Upload job triggers transcoding pipeline (Kafka event)
3. Transcoding service: convert to multiple resolutions (360p, 720p, 1080p, 4K)
4. Generate thumbnail
5. Update metadata DB: video available
6. CDN: pull-on-first-access for each PoP

Transcoding:

Expensive: 1 hour of video → 30 min transcoding (parallel)
AWS Elastic Transcoder / FFmpeg clusters
Adaptive Bitrate (ABR): HLS/DASH
Multiple quality levels, player switches based on bandwidth
Segment size: 2-10 seconds per segment

Streaming (adaptive bitrate):

Client: requests video → gets playlist (m3u8)
Playlist: list of segment URLs at different qualities
Client: monitors download speed → picks best quality
Segment: 5-10 second chunks → switch quality every segment

Data model:

videos: (video_id, uploader_id, title, description, duration, status, views)
video_files: (video_id, resolution, file_url, size)
comments: (video_id, user_id, text, created_at)
view_counts: ClickHouse (high write volume, analytics)

CDN strategy:

Popular videos: push to all CDN PoPs proactively
Long tail: pull CDN (fetch from origin on first request per PoP)
~20% of videos = ~80% of views (cache those aggressively)

Recommendation:

Collaborative filtering: users who watched A also watched B
Watch history + search history → embedding model → similarity
Pre-computed recommendations updated hourly (MapReduce / Spark)

View counter:

INCR in Redis → async batch write to DB every 5 minutes

Avoid DB write on every view (bottleneck at 1M QPS)

Interview tip: "Adaptive Bitrate (HLS/DASH) is the key YouTube/Netflix innovation. Without it: fixed quality → buffering on slow connections. With ABR: smooth experience regardless of bandwidth."

Q83. Design a social network feed (like Facebook News Feed).

Pattern:
Classic
design

Difficulty: Hard

Requirements:

Post content, follow people, home feed (ranked)
Non-functional: 1B users, 500M posts/day, <200ms feed load

Data model:

users: (user_id, name, email)
posts: (post_id, user_id, content, media_urls, created_at)
follows: (follower_id, followed_id)
feed: (user_id, post_id, score, created_at) [pre-computed]

Feed generation approaches:

Fan-out on write (push):

Alice posts → fanout worker → writes to feed table for all Alice's followers
Pros: read is $O(1)$ `SELECT * FROM feed WHERE user_id=? ORDER BY score DESC`
Cons: Alice has 100M followers → 100M writes (celebrity problem)

Fan-out on read (pull):

Read time: `SELECT * FROM posts WHERE user_id IN (followees) ORDER BY created_at DESC`
Pros: no write amplification
Cons: large followee lists → slow (join across millions of posts)

Facebook's hybrid:

Normal users: fan-out on write (push)
Celebrities (>10K followers): fan-out on read
At read time: merge pre-computed feed + celebrity posts

Feed ranking:

Score: recency + engagement (likes, comments, shares) + author relationship
EdgeRank (Facebook's original): $\text{Affinity} \times \text{Weight} \times \text{Time decay}$
Modern: deep learning model (trained on click-through rate)
Re-rank on every feed request (computationally expensive)
Pre-score: compute rough scores on write, fine-tune on read

Caching:

Top 1000 feed items per user in Redis list
Older posts: paginate from feed table in DB

Pagination:

Cursor-based: `?after=post_id_xyz` (not offset-based)
New posts inserted at top → offsets shift → cursor stable

Interview tip: "Facebook's EdgeRank is worth mentioning by name. The hybrid fan-out approach (push for normal, pull for celebrities) is the same as Twitter's. Both companies converged on this solution independently."

Q84. Design a chat application (like WhatsApp).Pattern:
Classic
design**Difficulty: Hard****Requirements:**

1:1 and group messaging, message delivery status, offline messages
Non-functional: 1B users, 100B messages/day, <100ms delivery

Estimation:

100B messages/day = 1.16M messages/sec

Architecture:

Client $\leftrightarrow$ WebSocket $\rightarrow$ Chat Server $\leftrightarrow$ Kafka $\leftrightarrow$ Chat Server $\leftarrow$ WebSocket $\rightarrow$ Client

Chat Server: maintains WebSocket connections

Message DB: store messages

User Service: manage user presence/status

Notification: push when recipient offline

Message flow (A $\rightarrow$ B):

1. Client A $\rightarrow$ WebSocket $\rightarrow$ Chat Server A
2. Chat Server A $\rightarrow$ write to Message DB + Kafka
3. If B online: Kafka consumer on Chat Server B $\rightarrow$ push via WebSocket to B
4. If B offline: Push notification (FCM/APNs)
5. B connects $\rightarrow$ pulls missed messages from Message DB

Message delivery status:

Sent (1 tick): server received message

Delivered (2 ticks): recipient's device received

Read (2 blue ticks): recipient opened conversation

Implemented via ACK messages from client

Data model:

messages: (message_id, chat_id, sender_id, content, type, created_at, deleted_at)

chat_members: (chat_id, user_id, joined_at, last_read_at)

Cassandra: time-series, high write, queries by (chat_id, time range)

message_id: Snowflake ID (time-ordered, globally unique)

Group messages:

Group $\rightarrow$ N members

Option 1: fan-out on write (write to each member's inbox) $\rightarrow$ simple but N writes

Option 2: store once, members read from group chat $\rightarrow$ efficient storage

WhatsApp: store once + fan-out for active members' caches

End-to-end encryption:

Signal protocol: each message encrypted with recipient's public key

Server never sees plaintext (can't comply with law enforcement easily)

WebSocket management:

Sticky sessions: consistent hash to same chat server per user_id

On reconnect: client sends last_message_id $\rightarrow$ server sends missed messages

Interview tip: "WebSocket for real-time + Kafka for fan-out + Cassandra for message storage is the production stack. Signal protocol for E2E encryption — mention it even if you don't implement it."

Q85. Design a payment processing system.Pattern:
Classic
design**Difficulty: Hard****Requirements:**

Charge card, refund, view transaction history

Non-functional: 99.999% availability, exactly-once processing, fraud detection

Key constraints:

Idempotency: network retries must not double-charge

ACID: transaction must be atomic (debit + credit)

Compliance: PCI-DSS (don't store raw card numbers)

Audit: every operation logged

Architecture:

```
Client → API Gateway → Payment Service → [PSP: Stripe/Braintree]
                                     → Ledger DB (immutable)
                                     → Fraud Detection Service
```

Idempotency:

Client sends Idempotency-Key: UUID with every payment request

Server: INSERT INTO idempotency_keys ... ON CONFLICT DO NOTHING

Same key → return stored response (no double charge)

Charge flow:

1. Client → POST /payments {amount, card_token, idempotency_key}
2. Check idempotency key (already processed?)
3. Fraud check (ML model: amount, merchant, device fingerprint)
4. Call PSP (Stripe): charge card token
5. On success: INSERT INTO ledger (amount, type=DEBIT, user_id, created_at)
6. Return success response + store in idempotency table

Double-entry bookkeeping:

Every transaction: one DEBIT + one CREDIT (must balance)

Ledger is append-only (never update/delete)

Balance = SUM of all entries for account

Failure handling:

PSP timeout: was the charge made? Check PSP status API (idempotent GET)

If unknown: return 202 Accepted → async check → notify via webhook

Outbox pattern:

Write to payment_events outbox in same DB transaction as ledger

Kafka publisher reads outbox → publishes PaymentProcessed event

Downstream: email, analytics, fraud update – all async

PCI-DSS compliance:

Never log raw card numbers

Use tokenization: card stored at Stripe, you store token only

Network segregation: payment service in separate VPC

Interview tip: "Idempotency + Outbox pattern + double-entry bookkeeping are the three payment system pillars. Mentioning PCI-DSS and tokenization shows operational awareness."

Q86. Design a distributed job scheduler (like Airflow/Cron).Pattern:
Classic
design**Difficulty: Hard****Requirements:**

Schedule jobs (cron or one-shot), execute, handle retries
Non-functional: 10K jobs, at-least-once, exactly-once with idempotent jobs

Components:

Scheduler: reads job definitions, triggers at right time
Job Store: DB of job definitions + execution history
Executor: executes jobs, reports status
Worker Pool: actual workers that run job code
Leader Election: only one scheduler instance runs at a time

Data model:

jobs: (job_id, name, cron_expr, handler, payload, max_retries, timeout, enabled)
job_runs: (run_id, job_id, scheduled_at, started_at, completed_at, status, error)

Scheduler loop (runs on leader only):

every second:
jobs_due = SELECT * FROM jobs WHERE next_run_at <= NOW() AND enabled=true
for each job: create job_run, enqueue to worker queue

Leader election:

Only one scheduler instance should run (prevent duplicate job runs)
Use DB SELECT ... FOR UPDATE or Redis SETNX
Renew every 30s; other schedulers wait

Worker execution:

Worker picks job from queue (Kafka or DB-backed queue)
Execute job handler
On success: UPDATE job_runs SET status='success', completed_at=NOW()
On failure: check retry count → retry (exponential backoff) or mark failed

Exactly-once semantics:

Check run_id in idempotency table before executing
Claim job: UPDATE job_runs SET status='running', worker_id=? WHERE status='pending'
If UPDATE affects 0 rows → another worker claimed it → skip

Distributed execution:

Workers can run on different machines
Job handler is a function registered by name
Worker fetches code or uses pre-deployed job containers

Timezone handling:

Store cron in UTC
Convert to user's timezone for display only
DST transitions: skip or repeat? (run once, usually)

Monitoring:

Job success rate dashboard
Stalled jobs alert: status='running' for > 2× timeout → mark as failed

Interview tip: "Leader election for the scheduler prevents duplicate executions. The atomic UPDATE ... WHERE status='pending' prevents two workers running the same job. These two together give at-most-once execution."

Q87. Design a hotel/flight booking system.Pattern:
Classic
design**Difficulty: Hard****Requirements:**

Search available rooms/flights, book, cancel, payment

Non-functional: no double booking, high availability during peak (holidays)

Double-booking prevention:

Critical: two users must not book the same seat/room simultaneously

Option 1: SELECT FOR UPDATE (pessimistic lock)

BEGIN

SELECT * FROM seats WHERE id=? AND status='available' FOR UPDATE

UPDATE seats SET status='booked', user_id=? WHERE id=?

COMMIT

Pro: safe; Con: high contention during peak

Option 2: Optimistic locking

SELECT seat, version FROM seats WHERE id=?

UPDATE seats SET status='booked' WHERE id=? AND version=?

If 0 rows updated → conflict, retry or fail

Option 3: Compare-and-swap at DB level

UPDATE seats SET status='booked', user_id=?

WHERE id=? AND status='available'

Check affected_rows == 1

Architecture:

Client → API → Inventory Service → DB (seats/rooms)

→ Payment Service (charge on success)

→ Notification (confirmation email)

Two-phase booking: hold → confirm/release (like airline seat hold)

Seat hold:

User selects seat → Hold for 10 minutes (soft lock)

If user pays → confirm hold → permanent booking

If 10 min expires → release hold → seat available again

Search optimization:

Pre-computed availability: cache available seat counts per flight

Real-time: DB for exact availability check at booking time

Elasticsearch for flexible search (date range, price range, destination)

Payment:

Charge AFTER hold confirmed

Release hold if payment fails

Outbox pattern: booking_confirmed → email/notification

Scale during peak:

Queue requests during spikes (Black Friday, Cyber Monday)

Show "virtual queue" to user

Process at steady rate

Interview tip: "The seat hold (two-phase) pattern is how airlines work. No airline actually charges you before confirming the seat. The SELECT FOR UPDATE + timeout is the core double-booking prevention."

Advanced System Designs

Q88. Design a distributed cache system (like Memcached cluster).

Pattern:
Advanced
design

Difficulty: **Hard**

Requirements:

Get, Set, Delete, TTL, distributed across N nodes
Non-functional: 1M QPS, <1ms p99, horizontal scale

Client-side sharding (consistent hashing):

Client maps key → node (no server-side routing)
libmemcached: consistent hashing built-in
Adding node: ~1/N keys remapped (vs naive modulo: all keys remapped)

Node architecture:

In-memory hash table (slab allocator – avoids fragmentation)
LRU eviction per slab class
Single-threaded (like Redis) or multi-threaded (Memcached: multi)

Slab allocator:

Memory divided into slab classes (64B, 128B, 256B, ...)
Each slab class: array of fixed-size chunks
No fragmentation (allocate whole chunk, not arbitrary size)
Trade-off: waste if item << chunk size

Facebook's Memcache paper insights:

Get/Set over UDP (faster, no connection overhead, best-effort)
TCP only for writes to primary (reliability)
Consistent hashing: minimal remapping on scale

Thundering herd prevention:

"stale-while-revalidate" mode: serve stale, refresh asynchronously
Lease system: only ONE client fetches from DB on cache miss
First client gets a "lease" (token); others back off and wait
If client fails, lease expires → next client can try

Replication:

Memcached: no built-in replication → client writes to multiple pools
For HA: write to pool A and pool B in parallel
Read: pool A only (replicated for DR, not primary HA)

Regional vs local caches:

Regional: shared cache across all app servers in region
Local: in-process cache (faster but not shared, inconsistent)
Both: local L1 (1-10K items, sub-ms) + regional L2 (100M items, 1ms)

Interview tip: "Facebook's Memcache paper (2013) is required reading for distributed caching. The lease system for thundering herd prevention is their key contribution. Mention it."

Q89. Design a real-time leaderboard.

Pattern:
Advanced
design

Difficulty: **Medium**

Requirements:

Update score, get rank, get top N, get neighbors of user
Non-functional: 10M players, real-time updates, <10ms queries

Redis Sorted Set (core data structure):

ZADD leaderboard {score} {player_id}	$O(\log N)$ insert/update
ZRANK leaderboard {player_id}	$O(\log N)$ rank lookup
ZREVRANGE leaderboard 0 99	$O(\log N + 100)$ top 100
ZSCORE leaderboard {player_id}	$O(1)$ score lookup
ZRANGEBYSCORE leaderboard min max	$O(\log N + M)$ range

Player's neighbors (nearby ranks):

rank = ZREVRANK leaderboard player_id
ZREVRANGE leaderboard (rank-5) (rank+5) WITHSCORES → neighbors

Update flow:

User scores points → ZADD leaderboard new_score player_id
Atomic: ZADD updates if score already exists

Multiple leaderboards:

"leaderboard:global"
"leaderboard:weekly:2024-w42"
"leaderboard:country:IN"
Separate Redis Sorted Sets

Time-bucketed leaderboards:

Weekly leaderboard: keys expire after 1 week
Redis ZADD leaderboard:weekly:{week} score player_id
TTL: EXPIREAT leaderboard:weekly:{week} {end_of_week_timestamp}

Large scale (>100M players):

Redis cluster: shard by player_id range or hash
But global rank requires merge across shards = complex
Alternative: approximate rank (good enough for most games)
Count players with score > mine = rank
Possible with segment tree or skip list approximation

Tie-breaking:

Same score: earlier achievers ranked higher
Compound score: $\text{actual_score} * 10^9 + (\text{MAX_TIME} - \text{timestamp})$
Monotonically decreasing → earlier time = higher rank

Interview tip: "Redis Sorted Set is the canonical answer for leaderboards. Enumerate all the operations with their time complexity. ZREVRANK in $O(\log N)$ is what makes it suitable for real-time use."

Q90. Design a web crawler.

Pattern:
Advanced
design

Difficulty: Hard

Requirements:

Crawl the web, extract URLs, store content, respect robots.txt
Non-functional: 1B pages, politeness, deduplication

Estimation:

1B pages, avg 100KB each = 100 TB
Rate: 100M pages/day = 1,160 pages/sec

Architecture:

URL Frontier → Fetcher → Parser → Link Extractor → Deduplicator → URL Frontier
→ Content Store

Components:

URL Frontier (priority queue):

- Stores URLs to crawl
- Priority: PageRank, freshness, importance
- Politeness: one queue per domain, delay between requests
- Redis sorted set per domain: (URL → priority)

Fetcher:

- HTTP client pool (100K concurrent connections)
- Respect robots.txt (cache per domain)
- Respect Crawl-delay in robots.txt
- DNS cache (don't resolve same host repeatedly)

Parser:

- HTML → extract links + text content
- Normalize URLs (lowercase, remove fragment, canonical form)

Deduplication:

- Bloom filter for visited URLs (avoid re-crawling)
- SimHash for near-duplicate content detection (same content, different URL)

Storage:

- Raw HTML: object store (S3)
- Parsed text + metadata: Elasticsearch
- URL graph: graph DB (for PageRank computation)

Scale-out:

- Multiple fetcher workers, partitioned by domain
- Consistent hash: domain → worker (politeness: only one worker per domain)

Re-crawl strategy:

- Freshness: high-value pages re-crawled daily (news), others monthly
- Change detection: check Last-Modified header or hash content

Interview tip: "The URL Frontier with per-domain queues + politeness delay is the core design challenge. Without politeness, you'll get IP-banned by every site. Bloom filter for dedup prevents re-crawling 1B visited URLs."

Q91. Design a stock trading system.

Pattern:
Advanced
design

Difficulty: Hard

Requirements:

- Place orders, match buy/sell orders, track portfolio, market data
- Non-functional: low latency (<1ms order matching), strict ordering, no data loss

Key concepts:

- Order book: sorted list of all open buy/sell orders per instrument
- Matching engine: matches buy orders against sell orders
- Market order: execute at best available price
- Limit order: execute at specified price or better

Order book structure:

- Bids (buyers): sorted descending by price

Asks (sellers): sorted ascending by price
Spread: ask - bid (tighter = more liquid)

Matching engine (critical path – ultra low latency):
Single-threaded (no locks!) for each trading pair
In-memory order book (hash map + sorted structure)
Persist to WAL before ACK
Match: highest bid $\geq$ lowest ask $\rightarrow$ trade

Order{type:LIMIT, side:BUY, price:100, qty:10}
Ask book has 100@ ask price $\rightarrow$ MATCH $\rightarrow$ trade

Data structures for order book:
Price levels: sorted map (price $\rightarrow$ total qty)
Orders at price: queue (FIFO for fairness)
Lookup by order_id: hash map

LMAX Disruptor pattern (used by trading firms):
Lock-free ring buffer for inter-thread communication
Single-writer principle
Cache-line padding (prevent false sharing)

Architecture:
Client $\rightarrow$ OMS (Order Management System) $\rightarrow$ Matching Engine
Matching Engine $\rightarrow$ Trade events $\rightarrow$ Kafka $\rightarrow$ Settlement, Risk, Market Data

Market data distribution:
Trades and order book updates $\rightarrow$ Kafka $\rightarrow$ Market Data Service
WebSocket feed to clients (sub-100ms)
Co-location: low latency clients physically locate servers near exchange

Risk checks (pre-trade):
Check buying power (available cash)
Position limits (can't hold $>X\%$ of stock)
Fat finger checks (order size within 10% of current market price)

Interview tip: "The matching engine must be single-threaded (no locks = predictable low latency). LMAX Disruptor is the production pattern. WAL ensures no order is lost. These three concepts cover the core of trading system design."

Q92. Design a fraud detection system.

Pattern:
Advanced
design

Difficulty: Hard

Requirements:
Detect fraudulent transactions in real-time ($<100\text{ms}$), block or flag
Non-functional: 100K TPS, low false positive rate, adaptive (learns from new patterns)

Architecture (two modes):
Synchronous (real-time): block transaction if fraudulent
Asynchronous (batch): detect patterns, update models

Real-time path ($<100\text{ms}$):
Transaction $\rightarrow$ Feature extraction $\rightarrow$ Rules engine $\rightarrow$ ML scoring $\rightarrow$ Decision
All steps in-process, in-memory where possible

Feature extraction:

Velocity features: 5 transactions in last 10 minutes? (Redis counter)
Location features: same card used in NY and London within 1 hour?
Behavioral: amount deviates from user's historical average?
Device: new device + new location + high amount = suspicious

Rules engine:

Fast explicit rules (no ML needed):
Rule 1: if amount > 10,000 AND new_device → flag
Rule 2: if country != user.home_country AND amount > 500 → flag
Rule 3: velocity > 5 transactions in 60s → block
Rules stored in DB, loaded on startup, hot-reloaded periodically

ML model:

Gradient boosted trees or neural network
Features: transaction features + user history features
Output: fraud probability (0-1)
Threshold: >0.7 → decline, 0.3-0.7 → 3D Secure challenge, <0.3 → allow
Model served via feature store + model server (Feast + Seldon)

Feedback loop:

Customer disputes → labeled fraud → retrain model
Chargeback data → ground truth
A/B test new models: 5% traffic to new model, compare performance

Redis for velocity checks:

INCR "velocity:{card_id}:{minute_bucket}"
EXPIRE 300 (5 minutes)
Atomic: read count + compare in Lua script

Graph analysis (async):

Find fraud rings: device → multiple accounts → same merchant
Graph DB: Neo4j or TigerGraph for entity relationship queries

Interview tip: "The synchronous + asynchronous dual-mode architecture is key. Real-time rules + ML for blocking, batch graph analysis for detecting rings. Feedback loop (disputes → retraining) shows systems thinking."

Q93. Design Google Docs (collaborative editing).

Pattern:
Advanced
design

Difficulty: Hard

Requirements:

Real-time collaborative editing, offline support, conflict resolution
Non-functional: <100ms sync latency, supports 100 concurrent editors per doc

Conflict resolution approaches:

Operational Transformation (OT) – Google Docs:

Each operation transformed against concurrent operations
If Alice inserts "X" at pos 5 and Bob inserts "Y" at pos 5:
Alice's insert is transformed to pos 6 for Bob's view (or vice versa)
Centralized server applies operations in order → source of truth
Server sends back transformed operations to all clients

CRDT (Conflict-free Replicated Data Types) – Figma, Notion:

Operations are inherently commutative (order doesn't matter)
No central coordination needed
Types: CRDT-text (RGA, LSEQ), CRDT-counter, CRDT-set

Better for offline-first / P2P, harder to implement efficiently

Architecture (OT-based like Google Docs):

Client → WebSocket → Doc Server → Redis (op queue) → DB (persisted ops)

Client:

Local buffer: apply operation immediately (optimistic UI)

Send to server: (op, base_revision)

Receive: (transformed_op, new_revision) → apply to local doc

Server:

Serialize all operations (single thread per document)

Transform incoming op against ops since client's base_revision

Apply to doc state

Broadcast transformed op to all other connected clients

Storage:

Full document: periodic snapshot to S3 (every N ops)

Operation log: Kafka (for replay) + DynamoDB (for queries)

Reconstruct doc: load last snapshot + replay ops since snapshot

Presence indicators:

Redis Pub/Sub: user cursor positions broadcast to all editors

Update every 100ms → smooth cursor movement

Offline support:

Buffer ops locally while offline

On reconnect: send all buffered ops with base_revision

Server: transform all buffered ops against concurrent ops → merge

Interview tip: "OT vs CRDT is a classic trade-off: OT requires central coordination (simpler for text, server is single source of truth); CRDT is P2P/offline-first (complex to implement correctly). Google chose OT, Figma chose CRDT."

Q94. Design an ads bidding/serving system.

Pattern:
Advanced
design

Difficulty: Hard

Requirements:

Match ads to user context, real-time bidding, click tracking, billing

Non-functional: 10M ad requests/sec, <10ms ad selection, accurate billing

Real-Time Bidding (RTB) flow:

Publisher page loads → ad request → Ad Exchange

Ad Exchange → OpenRTB request to all Demand-Side Platforms (DSPs)

DSPs bid within 100ms → highest bid wins → ad served

Ad serving (direct/owned inventory):

User context: location, device, browsing history, demographics

Ad selection: candidates → filtering → ranking → serve

Budget: don't serve ads from depleted campaigns

Targeting:

Audience segments: "25-34, male, interested in tech"

Contextual: keyword matching on page content

Behavioral: user visited electronics pages 3× this week

Retargeting: user added to cart but didn't buy

Ad auction (second-price / Vickrey):

Advertisers bid by CPM (cost per 1000 impressions) or CPC (per click)
 Winner: highest bid, pays second-highest price (+ 1 cent)
 Expected value: $\text{clicks} \times \text{CPC} \times \text{click-through-rate} = \text{expected revenue}$

Click fraud prevention:

Invalid Traffic (IVT) detection
 Check: same IP clicking many times, bot patterns, impossible click speed
 Machine learning model: probability this click is genuine

Billing:

Impression logged → Kafka → billing consumer → aggregate
 No immediate DB write on every impression (too slow at 10M/sec)
 Batch: aggregate by campaign per minute → write to billing DB
 Idempotency: deduplicate on (impression_id) before charging

Frequency capping:

User shouldn't see same ad >3 times/day
 Redis: INCR "freqcap:{user_id}:{ad_id}:{date}" → check < 3
 TTL: expire at midnight

A/B testing:

Test ad creative A vs B
 Random assignment (consistent hash by user_id)
 Measure CTR, conversion rate, revenue per impression

Interview tip: "Second-price auction is Google AdWords' original design. Know why: bidders bid their true value (dominant strategy) because they always pay less than their bid. This shows understanding beyond just the engineering."

Q95. Design an e-commerce platform (like Amazon).

Pattern:
Advanced
design

Difficulty: Hard

Requirements:

Browse products, search, add to cart, checkout, order tracking
 Non-functional: 100M DAU, peak 1M QPS (Prime Day), 99.99% availability

Services breakdown:

Product Catalog: browse, search (read-heavy, cacheable)
 Inventory: stock levels (write-heavy, consistency critical)
 Cart: add/remove items (session, eventually consistent)
 Order: place, track, cancel (ACID, event-driven)
 Payment: charge, refund (idempotent, audit)
 Search: full-text + filters (Elasticsearch)
 Recommendation: "also bought" (batch ML, served from cache)
 Notification: order updates (email/push, async)

Product catalog:

PostgreSQL for product data
 Elasticsearch for full-text search + faceted filters
 Cache: Redis/CDN for popular products (read at 1M QPS easily from cache)

Inventory management (hardest part):

Race condition: 1 item left, 1000 users trying to buy
 Solution: SELECT ... FOR UPDATE (DB pessimistic lock)
 Or: Redis DECR inventory:{product_id} → if < 0, rollback + out-of-stock

Cart:

Redis HASH: {user_id} → {product_id: qty, ...}
 TTL: 30 days (persistent for logged-in users)
 Merge carts on login (guest → authenticated)

Order state machine:

pending → payment_processing → confirmed → shipped → delivered → cancelled/returned

Flash sale (Peak load):

Cache inventory count in Redis
 Pre-sell: accept orders up to cached count
 Async: verify actual DB stock, cancel if oversold (compensate)
 Separate flash sale service with different limits

Recommendation engine:

"Customers also bought": collaborative filtering on purchase history
 Pre-computed hourly with Spark → stored in DynamoDB (product_id → [rec_ids])
 Read from cache → sub-millisecond

Search:

Elasticsearch: 1B products, fuzzy search, filters (price, brand, rating)
 Autocomplete: separate completion suggester index
 Personalized ranking: ML model boosts items user likely to buy

Interview tip: "Inventory management with race conditions is the hardest part. The SELECT FOR UPDATE pattern + Redis pre-check is the standard. For Black Friday scale: queue all add-to-cart operations and process sequentially per product."

Real-World Trade-offs

Q96. How do you choose between SQL and NoSQL for a specific use case?

Pattern:
Decision
framework

Difficulty:
Medium

Decision framework – ask these questions:

- What are the access patterns?
 "Get user by ID" → key-value lookup → DynamoDB excellent
 "Get all orders for user, joined with product details" → SQL
 "Time-series metrics, always append, range by time" → Cassandra/InfluxDB
- What consistency do you need?
 "User balance" → strong consistency → SQL with ACID
 "User feed" → eventual consistency → DynamoDB
- What is the scale?
 <10M rows/table, <10K QPS → SQL easily handles this
 >1B rows OR >100K QPS → evaluate NoSQL or SQL with sharding
- Does the schema change frequently?
 Stable schema (payments, users) → SQL, schema enforces correctness
 Rapidly changing (user preferences, config) → document DB
- Do you need complex queries?
 Reporting, aggregations, joins → SQL

Simple get/put by primary key → NoSQL

6. What's your team's expertise?

Most engineers know SQL → lower operational risk

NoSQL requires tuning knowledge (Cassandra: partition key, replication factor)

Concrete recommendations:

User accounts: PostgreSQL (ACID, joins, mature)

Product catalog: PostgreSQL + Elasticsearch for search

Shopping cart: Redis (TTL, fast, acceptable to lose)

Order history: PostgreSQL (ACID, audit)

User activity feed: Cassandra (time-series, write-heavy)

Sessions: Redis (TTL, low latency)

Analytics events: ClickHouse or BigQuery (columnar, high QPS analytics)

Relationships: PostgreSQL (FK constraints) or Neo4j (graph traversals)

Interview tip: "Never say 'NoSQL scales better and SQL doesn't.' Say: 'For this access pattern at this scale, DynamoDB is appropriate because X.' Always justify with access pattern + scale + consistency requirements."

Q97. How do you handle eventual consistency in your system?

Pattern:
Consistency
handling

Difficulty: Hard

Accepting eventual consistency requires understanding the user impact and designing for it.

Read-your-writes consistency:

Problem: User updates profile, immediately reads it → sees old value

Solution 1: Route user's reads to primary for 30s after write

HTTP header: X-Consistency: leader / routing table per user

Solution 2: Cache the write locally → serve from local cache for 30s

Solution 3: Read from primary always (sacrifices read scaling)

Monotonic reads:

Problem: Refresh page, see older data than previous request

Cause: routed to different replica with different lag

Solution: sticky sessions → same user always reads from same replica

Implementation: consistent hash(session_id) → replica

Causal consistency:

Problem: Alice replies to Bob's post before Bob sees the original post

Solution: vector clocks or logical timestamps

"If you've seen event X, you'll see all events causally before X"

Handling stale data in UI:

Optimistic updates: immediately update UI on write (assume success)

Rollback: if async confirmation fails, revert UI + notify user

"Your changes have been saved" → stale data risk acceptable

"Your payment was processed" → strong consistency required

Conflict resolution (when multiple writes to same data):

Last Write Wins (LWW): timestamp decides (risk: clock skew)

Multi-value (Dynamo): return all conflicting versions, app resolves

CRDT: mathematically conflict-free merge (Riak, Cosmos DB)

Practical rule:

User-visible state: read-your-writes minimum

Cross-user visible state (feed, counts): eventual consistency OK

Financial state: strong consistency mandatory

Interview tip: "The practical answer: 'We accept eventual consistency for the feed (user sees tweet 200ms later — fine). We require strong consistency for inventory (last item race condition — not fine). Design each component appropriately.'"

Q98. When do you choose microservices vs monolith?

Pattern:
Architecture
decision

Difficulty:
Medium

Monolith first (default for new products):

Advantages:

- Simple deployment (one binary)
- In-process communication (no network latency)
- Easier debugging (single process, single log)
- Simpler transactions (ACID across the whole app)
- Faster development (no service boundary overhead)
- Easier refactoring (all code visible)

When to stay monolith:

- Early-stage startup (product-market fit not found)
- Small team (<20 engineers)
- Low complexity domain
- Limited operational expertise

Microservices (when scale demands it):

Advantages:

- Independent deployments (teams ship without coordination)
- Independent scaling (scale payment service, not auth service)
- Technology heterogeneity (each service picks best tool)
- Fault isolation (user service crash doesn't kill payment service)
- Team ownership (each team owns one service end-to-end)

When to go microservices:

- >50 engineers (coordination overhead justifies it)
- Clear bounded contexts (DDD)
- Different scaling requirements per component
- Need independent release cadences
- Mature CI/CD + observability infrastructure

Costs of microservices:

- Network latency: function call (0ns) → network call (1-10ms)
- Distributed transactions: ACID → Saga + compensations
- Operational complexity: N services, N Kubernetes deployments, N alerts
- Testing: integration tests across services are complex
- Observability: need tracing across services (OpenTelemetry)

Migration path:

- Start monolith → identify bottlenecks → extract services one at a time
- Strangler Fig pattern: gradual extraction, not big-bang rewrite

Sam Newman's rule: "Don't start with microservices. Start with a monolith.

When the monolith's seams become clear, extract services along those seams."

Interview tip: "The correct answer for an early-stage startup is almost always 'start with a well-structured monolith.' Interviewers are testing whether you blindly jump to microservices or apply judgment. Apply judgment."

Q99. How do you handle database migrations at scale?Pattern:
Schema
evolution**Difficulty: Hard**

Challenge: change DB schema without downtime, while code is deployed.

Rules for zero-downtime migrations:

1. Schema changes must be backward compatible
2. Old code must work with new schema
3. New code must work with old schema (during rolling deploy)

Expand-Contract pattern (Parallel Change):

Step 1: Expand (add, don't remove)

```
ALTER TABLE users ADD COLUMN display_name VARCHAR(255)
```

Old code: ignores new column (OK)

New code: writes to new column

Deploy new code

Step 2: Migrate data

```
UPDATE users SET display_name = name WHERE display_name IS NULL
```

Run in small batches (avoid locking entire table)

```
Loop: UPDATE ... WHERE id BETWEEN ? AND ? AND display_name IS NULL
```

pg_repack: rewrite table without locks (PostgreSQL)

Step 3: Contract (remove old)

After new code is stable (weeks/months later)

```
ALTER TABLE users DROP COLUMN name
```

Old code already retired – no dependency

Deploy removal

Operations to avoid during live traffic:

`ALTER TABLE ADD COLUMN NOT NULL` (table lock in old PG) → add NULL first, then NOT NULL later

`DROP TABLE` – wrong table → catastrophic

Change column type (implicit full table rewrite) → add new column, migrate, rename

Blue-Green database:

Replication lag + dual-write during migration

Blue DB (old schema) → new writes to Green DB (new schema) via dual-write

Verify consistency → cut reads to Green

Stop dual-write

Tools:

golang-migrate: SQL migration files with up/down

Flyway, Liquibase: Java-origin, widely used

Atlas: Go-based, drift detection, cloud-native

gh-ost (GitHub): online schema change for MySQL (no locks)

pg_repack: online table reorganization for PostgreSQL

Interview tip: "The expand-contract pattern is the production answer for zero-downtime migrations. Adding NOT NULL columns or changing column types require special handling. Know gh-ost for MySQL or pg_repack for PostgreSQL."

Q100. How do you design a system for 10× current scale?Pattern:
Scalability
planning**Difficulty: Hard**

Framework: identify bottleneck, then apply appropriate technique.

Current state: 10K QPS, 100ms p99

Target: 100K QPS, 100ms p99 (10× scale, same latency)

Step 1: Identify bottlenecks with profiling

Application: CPU? Memory? Thread pool?

Network: bandwidth? connections?

Database: QPS limit? Lock contention? Slow queries?

Cache: hit rate? Evictions?

Step 2: Bottleneck → Solution mapping

CPU-bound application:

Horizontal scaling (add more servers)

Auto-scaling based on CPU metrics

Profile + optimise hot code paths

Database write bottleneck:

Connection pooling (PgBouncer)

Write batching (batch 100 inserts)

Async writes (queue → batch writer)

Sharding (if single-server limit hit)

CQRS (separate read/write paths)

Database read bottleneck:

Read replicas (distribute read traffic)

Caching (Redis) for frequent queries

Query optimisation (EXPLAIN ANALYZE)

Denormalize (avoid expensive JOINS)

Search off-load to Elasticsearch

Cache bottleneck:

Redis Cluster (shard cache)

Increase cache size

Optimize key design (avoid large keys)

Network bottleneck:

CDN for static assets

Compression (gzip/brotli)

Connection pooling

HTTP/2 multiplexing

Application state bottleneck:

Stateless services (move state to Redis)

Enables unlimited horizontal scaling

10× doesn't always mean 10× cost:

Caching + CDN: 10× traffic at 2× cost

Algorithmic optimisation: 10× capacity at same cost

Sharding: 10× capacity at 10× cost (last resort)

Priority: optimise → cache → scale out → shard

Interview tip: "The best answer to '10× scale' shows a systematic approach: profile, identify bottleneck, apply targeted solution. Don't blindly add more servers — that only helps CPU-bound services. The database is almost always the bottleneck at scale."

Quick Reference: HLD Decision Cheat Sheet

Decision	Default Choice	Switch When
DB (relational)	PostgreSQL	>1B rows → shard or DynamoDB
DB (document)	MongoDB	Simple key-value → DynamoDB
Cache	Redis	>100GB or >1M ops/s → Redis Cluster
Message queue	Kafka	Simple task queue → SQS/RabbitMQ
Search	Elasticsearch	Simple FTS → PostgreSQL FTS
CDN	CloudFront/Cloudflare	Always use for static assets
Load balancer	AWS ALB	Bare metal → HAProxy/Nginx
Storage	S3	Always for blobs
Service discovery	Kubernetes DNS	Complex routing → Consul
Tracing	OpenTelemetry	Always instrument with OTel

Quick Reference: Number Cheatsheet

Metric	Value
1 day	~86,400 seconds (~100K for estimation)
1M users, 1 req/day	~12 QPS
Single DB server	~10K QPS (simple queries)
Redis	~1M ops/sec (single node)
Kafka	~1M msgs/sec (single broker)
S3 latency	~20-100ms first byte
CDN latency	5-20ms edge hit
SSD sequential read	3-5 GB/s
Network (same AZ)	<1ms
Network (cross-AZ)	1-5ms
Network (cross-region)	50-150ms
3 nines availability	8.77 hours downtime/year
4 nines availability	52.6 min downtime/year

*Good luck with your SDE2 HLD interviews! Draw diagrams, state trade-offs, and always quantify your decisions.

■*

*Push to GitHub: `git add . && git commit -m "Add 100 HLD SDE2 interview questions" && git push*`

HLD Additional Questions (Q101–Q150)

Q101. How would you design a URL shortener (Bit.ly)?

Difficulty: Hard

Requirements:

- Shorten URL → unique 6-7 char code
- Redirect shortened → original URL
- Analytics: click count, geography
- Scale: 100M URLs, 10B redirects/day

Capacity:

- Write: $100M \text{ URLs} / (365 \times 86400) \approx 3 \text{ writes/sec}$ (trivial)
- Read: $10B / 86400 \approx 115K \text{ reads/sec}$ (high read)
- Storage: $100M \times 500 \text{ bytes} = 50GB$

API:

- POST /shorten → {short_code: "abc123"}
- GET /{code} → 301/302 redirect

Short code generation:

- Option 1: hash(long_url) → take 6 chars (collision risk)
- Option 2: auto-increment ID → base62 encode
 - ID=1 → "1", ID=62 → "10" ... ID=56800235584 → "ZZZZZZ"
 - 6 chars of base62 = $62^6 = 56 \text{ billion URLs}$
- Option 3: random 6 chars (must check uniqueness)

Architecture:

- API servers (stateless, scale horizontally)
- DB: PostgreSQL for URL mappings (sharded by short_code)
- Cache: Redis (LRU, 90% cache hit for popular URLs)
 - Cache: shortCode → longURL (100 million active codes)
- CDN: cache redirects at edge (301 = permanent, cached by browser)

Redirection:

- 302 (temporary): always hits server, accurate analytics
- 301 (permanent): browser caches, fewer server hits but no analytics

Counter service (analytics):

- Don't count in hot path (slows redirect)
- Emit click event → Kafka → analytics service → update counts async

Database schema:

- urls: id, short_code, long_url, user_id, created_at, expires_at
- clicks: short_code, timestamp, ip, country, user_agent

Q102. How would you design a ride-sharing system (Uber)?

Difficulty: Hard

Requirements:

- Match riders to nearby drivers
- Real-time location tracking
- Dynamic pricing (surge)
- ETA calculation
- Payment processing

Key challenges:

- Location updates: 10M drivers, update every 4 seconds
- Matching: find closest available driver in <1 second
- Consistency: prevent one driver matched to two riders

Location service:

- Geospatial index: QuadTree or H3 (Uber's hexagonal grid)
- Redis Geo: GEOADD/GEORADIUSBYMEMBER for nearby search
- Location update: driver app → WebSocket → location service → Redis

Driver matching:

- Rider requests ride → supply matching service
- Query Redis Geo for drivers within R km
- Filter: available, car type matches
- Sort by: estimated pick-up time
- Assign: optimistic locking or distributed lock (Redis SETNX)

Surge pricing:

- Supply/demand ratio per H3 cell
- demand > supply × threshold → surge multiplier
- Recalculate every minute

Payment:

- Trip completion → async payment request
- Stripe/payment processor
- Retry with exponential backoff

Microservices:

- Rider service, Driver service, Matching service
- Trip service, Payment service, Notification service
- Location service (high throughput, separate)

Real-time comms:

- Driver/rider location: WebSocket → location service
- Notifications: Firebase Cloud Messaging

Data:

- Trips: PostgreSQL (ACID, history)
- Location: Redis (in-memory, evict old)
- Analytics: Kafka → Flink → BigQuery

Q103. How would you design a distributed message queue?**Difficulty: Hard****Requirements:**

- Producers publish messages
- Consumers receive messages in order
- At-least-once delivery
- Scale: 1M messages/sec, 1KB avg size

Key components:

- Broker: stores messages
- Topic: logical channel
- Partition: ordered sub-channel per topic
- Consumer group: set of consumers sharing work

Storage:

- Append-only log files (like Kafka)

Each partition = single log file
Index: offset → file position (O(1) seek)
Retention: time-based or size-based

Replication:

Leader-Follower: 3 replicas
Leader: handles all reads/writes
Followers: replicate from leader (async or sync)
Leader election: ZooKeeper/Raft
acks=all: wait for all replicas (durability)

Producer:

Hash(key) % partitions → choose partition
Batch + compress for throughput
Retry on failure (idempotent with sequence numbers)

Consumer:

Pull model (consumer controls pace)
Long-poll for efficiency
Offset tracking: commit after processing
Rebalance: partition reassignment on consumer join/leave

Delivery guarantees:

At-most-once: commit offset before processing (fast, may lose)
At-least-once: commit after processing (may duplicate)
Exactly-once: transactions (producer + consumer atomically)

Throughput optimization:

Batch: multiple messages per request
Compression: lz4/snappy
Zero-copy: sendfile() syscall (disk → NIC, no user space)
Page cache: OS file cache for recent messages

Q104. How would you design a video streaming platform (YouTube)?**Difficulty: Hard****Requirements:**

Upload videos (large files, up to 4K)
Stream videos to millions of concurrent viewers
Search, recommendations
Scale: 500 hours uploaded/minute, 1B daily views

Video upload pipeline:

Client → chunked upload (resumable, Google TUS protocol)
Object storage (S3) → raw video stored
Message queue (SQS/Kafka) → trigger processing
Video processor (FFmpeg):
 Transcode: 4K → 1080p, 720p, 480p, 360p
 Extract thumbnail at 10%
 Generate HLS segments (10-second chunks)
Processed segments → CDN origin

Storage:

Metadata: PostgreSQL (title, description, views, user_id)
Raw + transcoded: S3 (cost: \$0.023/GB/month)
Thumbnails: S3 + CDN

CDN strategy:

Popular videos: Cloudflare/CloudFront at 200+ PoPs
First play: cold start from origin, CDN caches
Popular (top 1%): proactively pushed to CDN
Long-tail (99%): served from nearest region or on-demand

Adaptive bitrate (ABR):

HLS/DASH: client switches quality based on bandwidth
Player monitors buffer health, requests appropriate segment
720p → pause/buffer → switch to 480p automatically

Recommendations:

Collaborative filtering (matrix factorization)
Two-tower DNN: user tower + video tower
Near real-time: offline model (daily) + online feature store
Candidate generation → ranking → reranking (diversity)

Search:

Video metadata → Elasticsearch
Autocomplete: Redis prefix cache
Trending: Kafka streams, top-K per time window

Q105. How would you design Instagram (photo sharing)?

Difficulty: Hard

Requirements:

Upload photos/videos
Follow users, see feed
Likes, comments
Direct messages
Notifications

Feed generation (hardest problem):

Push model: on post → write to all followers' feeds
Pros: read is cheap $O(1)$
Cons: write is expensive (celebrity with 100M followers)

Pull model: on read → fetch from all followings
Pros: write is cheap
Cons: read expensive (merge N timelines)

Hybrid (Instagram's approach):

Regular users (< 10K followers): push
Celebrities (> 10K): pull at read time
Reader merges: cached feeds + celebrity posts

Timeline storage:

Redis sorted set: {user_id:feed → [(timestamp, post_id), ...]}
Keep last 200 posts in cache
On cache miss: query activity DB

Post storage:

Photos: S3 + CDN (multiple sizes: thumbnail, medium, full)
Post metadata: PostgreSQL (post_id, user_id, caption, location)

Social graph:

Follows table: (follower_id, following_id, created_at)
Sharded by user_id
Cassandra or PostgreSQL (large scale → Cassandra)

Notifications:

- Like/comment/follow → event → notification service
- Push: Firebase (mobile), Web Push
- In-app: WebSocket or long-poll
- Batch: digest emails (daily summary)

Direct messages:

- Cassandra for message storage (time-series, append-heavy)
- WebSocket for real-time delivery
- Offline: push notification when recipient connects

Q106. How do you design for global scale (multi-region)?**Difficulty: Hard****Multi-region architecture:****Active-Active (two primary regions):**

- Users routed to closest region (GeoDNS/Anycast)
- Data replicated between regions (async)
- Conflict resolution needed for concurrent writes
- Best for: global users, low latency required

Active-Passive (primary + DR):

- All writes to primary
- Passive region: hot standby, replicates from primary
- Failover: DNS cutover (60s-5 minutes)
- Best for: compliance, simpler consistency model

Data strategies:

- Partition by geography: EU users → EU DB only
- No cross-region sync needed, GDPR-friendly
- Problem: users traveling need cross-region access

Global tables (DynamoDB Global Tables, CockroachDB):

- Multi-region active-active
- Eventual consistency across regions
- Conflict-free replicated data types (CRDTs)

Latency considerations:

- Speed of light: NYC → London = 70ms (min)
- Synchronous cross-region write: 70ms added to every write
- Solution: async replication (accept eventual consistency)

Consistency models:

- Strong: all regions see same value (high latency)
- Eventual: regions catch up eventually (low latency, may read stale)
- Causal: read your writes guarantee (common solution)
 - Read from region where you wrote
 - Or: read timestamp/token passed along

CDN for global reach:

- Static assets: CDN (global cache, <10ms)
- API: closest region (GeoDNS)
- DB reads: replica in closest region
- DB writes: home region (or nearest write replica)

Q107. How would you design a search autocomplete system?**Difficulty: Hard**

Requirements:

As user types → suggest completions
Personalized (user history) or global (popular)
Latency: <100ms

Approaches:**1. Trie (prefix tree):**

Each node = character
Path from root = prefix
Each node stores top-K completions (cached)
 $O(L)$ lookup where L = prefix length
Problem: storage, hard to distribute

2. Redis + Sorted Sets:

score = frequency of search term
`ZRANGEBYLEX "search:" "[query" "[query\xFF" LIMIT 0 5`
Very fast, scales well, simple
Problem: doesn't handle typos, no ranking by click-through

3. Elasticsearch prefix query:

match_phrase_prefix or edge_ngram tokenizer
Handles typos (fuzzy match)
Richer features (spell correction, synonyms)
Latency: ~20ms (acceptable)

4. Production hybrid:

Redis: top 10K global searches (fast, hot)
Elasticsearch: full search index (fallback, typo tolerance)
User history: personalized suggestions (Redis per-user sorted set)

Architecture:

Search typed → API → L1 cache (in-process, 10ms TTL)
→ L2 Redis (1ms, top terms)
→ Elasticsearch (20ms, full index)

Analytics pipeline:

User searches → Kafka → Flink → update frequency scores
Recalculate top-K per prefix → update Redis + Trie

Weight function:

$\text{score} = \log(\text{frequency}) \times \text{click_through_rate} \times \text{recency_boost}$

Q108. How would you design a rate limiter for an API gateway?**Difficulty: Hard****Requirements:**

Limit requests per user/IP/API key
Multiple windows: 100 req/min AND 1000 req/hour
Distributed (multiple API gateway instances)
Low latency: <5ms for rate limit check

Algorithm comparison:

Fixed window: simple, boundary burst problem
Sliding window log: accurate, high memory (store timestamps)
Token bucket: burst allowed, most common for APIs
Sliding window counter: good balance (approximate sliding window)

Distributed implementation (Redis):

Sliding window with Redis:

Key: `rate_limit:{user_id}:{window_start}`
 INCR + EXPIRE per window

```
def is_allowed(user_id, limit, window_sec):
    now = time.time()
    window_key = f"rate:{user_id}:{int(now/window_sec)}"
    count = redis.incr(window_key)
    if count == 1:
        redis.expire(window_key, window_sec * 2)
    return count <= limit
```

Token bucket (Lua script, atomic):

```
KEYS[1] = user key
current_tokens, last_refill = GET(KEYS[1])
elapsed = now - last_refill
new_tokens = min(capacity, current_tokens + elapsed * rate)
if new_tokens >= 1: SET(KEYS[1], new_tokens - 1, now); return ALLOW
else: return DENY
```

Headers to return:

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1234567890 (Unix timestamp when resets)
Retry-After: 30 (seconds to wait, on 429)
```

Tiered rate limits:

```
Anonymous: 10 req/min
Free tier: 100 req/min
Pro tier: 1000 req/min
Enterprise: unlimited (or very high)
```

Q109. How would you design a real-time collaborative document editor (Google Docs)?

Difficulty: Hard

Requirements:

- Multiple users edit same document simultaneously
- Changes appear in real-time for all collaborators
- No conflicts, correct merging
- Offline support

Concurrency approaches:

1. Operational Transformation (OT):

- Original Google Docs approach
- Each operation transformed against concurrent ops
- $\text{transform}(\text{op1}, \text{op2}) \rightarrow \text{op1}'$ that works after op2 applied
- Complex to implement correctly

2. CRDTs (Conflict-free Replicated Data Types):

- Each character has unique ID (author + counter)
- Operations: insert, delete (with tombstone)
- Any order of ops $\rightarrow$ same final state
- Yjs, Automerge libraries

3. Event sourcing:

- Every keystroke = event
- Events applied in order
- Replay to get document state

Architecture:

- Client: local in-memory document state
- WebSocket: bidirectional ops stream
- Collaboration server: fan-out ops to other connected clients
- Database: store document + op log

Conflict resolution (OT example):

- User A at pos 5 inserts "X"
- User B at pos 5 inserts "Y" (same position!)
- A's op applied first
- B's op transformed: insert at pos 6 (after X)
- Result: "...XY..."

Presence (cursor/selection):

- Each user's cursor position broadcast via WebSocket
- Store in Redis (ephemeral, TTL 30s)
- Display other users' cursors with their color/name

Offline sync:

- Client queues ops while offline
- On reconnect: send queued ops with last-seen version
- Server replays and transforms conflicting ops

Q110. How do you design database sharding?

Difficulty: Hard

Sharding: split data across multiple DB instances

Sharding strategies:

1. Range sharding:

- Users 1-1M → Shard 1
- Users 1M-2M → Shard 2
- Simple, enables range scans
- Problem: hot shards (new users always on last shard)

2. Hash sharding:

- $\text{shard} = \text{hash}(\text{user_id}) \% N$
- Even distribution
- Problem: range queries span all shards, resharding hard

3. Directory sharding:

- Lookup table: $\text{user_id} \rightarrow \text{shard}$
- Most flexible (any distribution, easy to move)
- Problem: lookup table is single point of failure (cache it!)

4. Geo sharding:

- EU users → EU shard
- US users → US shard
- Good for: data locality, compliance (GDPR)

Cross-shard problems:

- Joins: must query all shards, merge in application

Aggregations: partial agg per shard, merge in app
Unique IDs: use distributed ID generator (Snowflake, UUID)
Transactions: 2-phase commit (slow) or eventual consistency

Resharding (hard):

Add shard → move data from overloaded shard
Consistent hashing: minimize data movement when adding nodes
Double-write during migration: write to old + new shard
Gradually migrate reads to new shard

Alternatives to manual sharding:

CockroachDB, Spanner: auto-sharding
Vitess: MySQL sharding proxy (used by YouTube)
Citrus: PostgreSQL sharding extension

Q111. ### Q111–Q150: Additional HLD Questions

Difficulty:
Medium

Q112. How would you design an email system?

Difficulty:
Medium

Components: SMTP relay, mailbox storage, spam filter, MX records
Scale: billions of emails/day (Gmail scale)
Storage: append-only, IMAP protocol for access
Key: deduplication, spam detection, attachment handling

Q113. How would you design a payment processing system?

Difficulty:
Medium

Idempotency: unique transaction ID prevents double charges
Two-phase: authorize → capture
Rollback: compensating transactions
PCI-DSS: tokenization, never store raw card numbers
Queue: async processing for high volume

Q114. How would you design a hotel booking system?

Difficulty:
Medium

Inventory: rooms × dates matrix
Pessimistic locking during checkout (prevent overbooking)
Two-phase commit: reserve → confirm + payment
Read replicas for search, primary for booking

Q115. How would you design Twitter's trending topics?

Difficulty:
Medium

Count tweets per hashtag in rolling 1-hour window
Approximate top-K with Count-Min Sketch
Distributed counters with Kafka Streams
Geo-segmented trends (per country/city)

Q116. How would you design a recommendation engine?

Difficulty:
Medium

Collaborative filtering: similar users → similar items
Content-based: item features → similar items

Matrix factorization: ALS/SVD for embeddings
Two-tower model: user tower + item tower
Candidate generation → ranking → reranking

Q117. How would you design a flash sale system?**Difficulty:**
Medium

High concurrency write: inventory decrement
Pre-warm cache: product info in Redis before sale
Queue: serialize purchase requests (SQS FIFO)
Limit: per-user rate limiting
Inventory: atomic DECR in Redis, sync to DB async

Q118. What is the saga pattern for distributed transactions?**Difficulty:**
Medium

Choreography: each service reacts to events (decentralized)
Orchestration: central orchestrator coordinates (centralized)
Compensating transactions: undo actions on failure
State machine: track saga progress
Idempotency: operations safe to retry

Q119. What is the event sourcing pattern?**Difficulty:**
Medium

Store events, not state
Current state = replay of events
Commands → events → state projections
Benefits: audit log, time travel, event replay
Challenges: eventual consistency, query complexity
CQRS pair: commands to event store, queries to projection

Q120. What is CQRS (Command Query Responsibility Segregation)?**Difficulty:**
Medium

Commands: write path (DB with ACID)
Queries: read path (optimized read models)
Sync: commands update read models via events
Benefits: scale reads independently, optimize per use case
Trade-off: eventual consistency between write and read models

Q121. How would you design a distributed cache?**Difficulty:**
Medium

Consistent hashing: distribute keys across nodes
Replication: 2-3 replicas per key
Eviction: LRU/LFU policies
Invalidation: TTL + explicit delete on DB write
Thundering herd: singleflight, jitter on TTL

Q122. How would you design a job scheduling system?**Difficulty:**
Medium

Job types: one-time, recurring (cron), delayed
Storage: jobs table with scheduled_at, next_run, status
Polling: workers SELECT FOR UPDATE SKIP LOCKED

Partitioning: by scheduled_at (time-based)
Distributed: each worker picks up jobs independently (no coordinator)

Q123. What is the two-phase commit (2PC)?**Difficulty:**
Medium

Phase 1 (Prepare): coordinator asks all participants "ready?"
Participants: execute tx, write to undo log, send READY
Phase 2 (Commit/Abort): coordinator sends final decision
If all READY: COMMIT
If any FAIL: ABORT (all execute undo log)

Problem: coordinator crashes between phases → participants blocked
Improvement: 3PC, Paxos, Raft
Alternative: saga pattern (eventual consistency)

Q124. What is consistent hashing with virtual nodes?**Difficulty:**
Medium

Ring: 0 to $2^{32} - 1$
Nodes placed at M virtual positions (150 each)
Key: hash → ring → clockwise to next node
Adding node: only 1/N keys remap
Removing node: only keys from removed node remap
Virtual nodes: ensure uniform distribution even with few physical nodes

Q125. How would you design a leaderboard system?**Difficulty:**
Medium

Redis Sorted Set: ZADD leaderboard score user_id
ZRANK: position ($O(\log N)$)
ZREVRANGE: top N ($O(\log N + N)$)
ZINCRBY: update score atomically

Sharding: multiple sorted sets for categories
Periodic: recalculate from source of truth (daily/hourly)

Q126. What is a bloom filter in system design?**Difficulty:**
Medium

Probabilistic: "definitely not in set" OR "possibly in set"
Space-efficient: 10 bits per element (1% false positive rate)
Use cases:

- Cache: check if key worth fetching from DB
- Duplicate detection: has this URL been crawled?
- Spam filter: has this email been blacklisted?
- Username availability check (pre-filter before DB)

Q127. How would you design a log aggregation system?**Difficulty:**
Medium

Agents (Filebeat/Fluentd) → Kafka → Processors → Elasticsearch
Structured logging (JSON) at source
Log levels: ERROR (alert), WARN (monitor), INFO (standard)
Retention: hot (Elasticsearch 7d) → warm (S3 30d) → cold (Glacier 1y)
Alerting: threshold on error rate → PagerDuty

Q128. What is eventual consistency and how to handle it?**Difficulty:**
Medium

Eventual consistency: replicas converge after some delay
Read-your-writes: user sees own changes immediately
→ Read from same region where you wrote
→ Session token with write timestamp (read replica catches up)
Monotonic reads: never read older state than before
→ Route user to same replica
Bounded staleness: guarantee convergence within T seconds
Conflict resolution: last-writer-wins, version vectors, CRDTs

Q129. How would you design a live sports scoring system?**Difficulty:**
Medium

Write path: official score update → API → Kafka → processors
Read path: WebSocket fan-out to millions of concurrent viewers
Cache: current scores in Redis (TTL = match active)
Architecture: publish-subscribe with WebSocket servers
Scale: Kafka partitioned by match_id, WebSocket per server cluster

Q130. What is the strangler fig pattern for migration?**Difficulty:**
Medium

Gradually replace legacy system
Route % of traffic to new system
Increase % as confidence grows
Feature flag: A/B test old vs new
Dual write: write to both during migration
Read from new if available, fall back to legacy
Cut over: all traffic to new, legacy becomes fallback

Q131. What is back-pressure in distributed systems?**Difficulty:**
Medium

Back-pressure: consumer signals producer to slow down
Channel-based (Go): producer blocks when channel full
Queue-based: queue depth triggers producer to wait
Rate limiting: producer respects consumer's rate
Circuit breaker: stop sending when downstream overloaded
Load shedding: drop low-priority requests under high load

Q132. How would you design database migration in production?**Difficulty:**
Medium

Zero-downtime migration steps:
1. Add new column (nullable, no default needed yet)
2. Dual-write: write to old + new column
3. Backfill: update existing rows in batches
4. Read from new column
5. Drop old column

Online schema change tools:
gh-ost (GitHub): shadow table + triggers
pt-online-schema-change: copy + triggers
pg_repack: PostgreSQL table rewrite without locks

Q133. What is blue-green deployment?**Difficulty:**
Medium

Two identical production environments: blue (active) and green (idle)
Deploy to green → test green → switch traffic to green
Blue becomes idle (rollback target)
Benefits: zero-downtime, instant rollback
Costs: 2x infrastructure
DNS/load balancer: switch routing, not infrastructure

Q134. What is canary deployment?**Difficulty:**
Medium

Gradually roll out to subset of users/servers
5% traffic → canary servers with new version
Monitor: errors, latency, business metrics
Good? → 25% → 50% → 100%
Bad? → roll back (only 5% affected)

Canary selection:

- By server: route 5% of servers to new version
- By user: ring-based (internal → beta → prod)
- By header: X-Canary: true for opt-in

Metrics to monitor:

- Error rate (HTTP 5xx), P99 latency, custom business metrics

Q135. What is feature flags in system design?**Difficulty:**
Medium

Feature flags: toggle features without deployment

Types:

- Release: gradually roll out feature
- Experiment: A/B test
- Ops: emergency kill switch
- Permission: specific users/orgs

Storage:

- DB: feature_flags table (flag_name, enabled, rollout_percent)
- Config: LaunchDarkly, Unleash, Flagsmith, homegrown
- Cache: Redis with TTL (not every request hits DB)

Implementation:

```
if featureFlags.IsEnabled("new-checkout", userID):  
    newCheckout()  
else:  
    oldCheckout()
```

Q136. What are SLIs, SLOs, and SLAs?**Difficulty:**
Medium

SLI (Service Level Indicator): what you measure
Availability: $\text{successful_requests} / \text{total_requests}$
Latency: $P99 < 200\text{ms}$
Error rate: $5xx / \text{total} < 0.1\%$

Throughput: requests/second

SLO (Service Level Objective): target for SLI

Availability: 99.9% per month

Latency P99: < 200ms

Error rate: < 0.1%

SLA (Service Level Agreement): contractual commitment (SLO with consequences)

99.9% uptime → customer gets credit if violated

Error budget: 100% - SLO = allowed downtime

99.9% → 43.8 minutes downtime/month budget

Spend budget on: deployments, experiments, risky changes

Budget exhausted → freeze risky changes, focus on reliability

Q137. What is the two generals problem?

Difficulty:
Medium

Two generals must coordinate attack, communicate over unreliable channel

Problem: neither can be certain the other received the message

Even with unlimited retries: no guarantee of synchronized action

TCP: similar problem → uses sequence numbers + ACK (imperfect)

Real systems: accept uncertainty, use timeouts + retries

Distributed consensus (Paxos/Raft): solve with majority quorum

$2f+1$ nodes needed to tolerate f failures

Q138. What is the CAP theorem?

Difficulty:
Medium

CAP: Consistency, Availability, Partition Tolerance

Theorem: distributed system can guarantee only 2 of 3

Partition Tolerance: always required (networks fail)

So real choice is: CP or AP

CP (Consistent + Partition Tolerant):

Sacrifice availability during partition

Examples: ZooKeeper, HBase, etcd

Use: coordination, leader election, banking

AP (Available + Partition Tolerant):

Sacrifice consistency during partition (eventual consistency)

Examples: Cassandra, DynamoDB, CouchDB

Use: shopping carts, social feeds, DNS

PACELC extension:

PAC: partition → choose A or C

ELC: no partition → choose Latency or Consistency

DynamoDB: PA/EL (available under partition, eventual consistency for latency)

CockroachDB: PC/EC (consistent, accepts higher latency)

Q139. What is API pagination patterns?

Difficulty:
Medium

1. Offset pagination: ?page=3&limit=20

Simple, supports random access

Problem: items shift when new ones added (inconsistent pages)

N+1 queries: COUNT(*) for total pages

2. Cursor pagination: ?after=eyJpZCI6NDJ9&limit=20
 Cursor encodes: last seen ID (or timestamp+ID)
 Consistent: insertions don't affect cursor position
 Cannot jump to arbitrary page
 Best for: feeds, infinite scroll
3. Keyset pagination: ?min_id=42&limit=20
 Use indexed column directly (no encode/decode)
 SELECT * WHERE id > 42 ORDER BY id LIMIT 20
 Fast (index seek, no offset scanning)
 Cannot go backwards (unless bi-directional)

Choose:

Admin lists: offset pagination (need to jump to page 50)
 Infinite scroll feeds: cursor pagination
 High volume APIs: keyset pagination (most efficient)

Response format:

```
{
  "data": [...],
  "pagination": {
    "next_cursor": "abc123",
    "has_more": true
  }
}
```

Q140. How would you design a distributed lock?

Difficulty:
Medium

Redis Redlock (recommended):

N Redis nodes (5 for production)
 Acquire: SETNX key client_id EX ttl on majority (N/2 + 1)
 Release: check client_id matches, then DEL (Lua script)
 Valid if: acquired majority AND elapsed < TTL

Properties:

Safety: at most one holder at a time
 Liveness: lock eventually released (TTL)
 Fault tolerance: majority quorum survives node failures

go-redis implementation:

```
import "github.com/bsm/redislock"
locker := redislock.New(rdb)
lock, err := locker.Obtain(ctx, "order:42", time.Second*10, nil)
defer lock.Release(ctx)
```

Alternative: etcd / ZooKeeper (stronger guarantees)

etcd: distributed key-value with strong consistency
 lease: automatic expiry when client disconnects
 Sequential node: first one is leader

Fencing token: monotonically increasing token with lock

Storage checks: only accept writes with higher fencing token
 Prevents stale lock holder from corrupting state

Q141. What is event-driven architecture?**Difficulty:**
Medium

Events: immutable facts about what happened
 "OrderPlaced", "PaymentProcessed", "InventoryUpdated"

Patterns:

Event notification: notify other services (lightweight, no payload)
 Event-carried state transfer: full payload (no callback needed)
 Event sourcing: events as source of truth

Choreography vs Orchestration:

Choreography: services react to events independently
 Pros: loose coupling, no central point of failure
 Cons: hard to track workflow, emergent behavior

Orchestration: central coordinator (Step Functions, Temporal)
 Pros: explicit workflow, easy to trace
 Cons: central coupling

Message brokers:

Kafka: high throughput, replay, multiple consumers
 RabbitMQ: routing, patterns, acknowledgment
 EventBridge: AWS-native, routing rules

Challenges:

Ordering: per partition only
 Exactly-once: use idempotency keys
 Schema evolution: backward/forward compatible
 Dead letter handling: after N retries
 Observability: distributed tracing across events

Q142. ### Q141–Q150: Final HLD Questions**Difficulty:**
Medium

Q	Topic
Q141	Design a distributed file system (HDFS-like)
Q142	Design a social graph (Facebook friend recommendations)
Q143	Design a real-time bidding system
Q144	Design a content moderation system
Q145	Design a distributed session management
Q146	How to handle data migration at scale
Q147	Design an IoT data ingestion pipeline
Q148	Design a metrics and monitoring system
Q149	Multi-tenancy: shared vs isolated database per tenant
Q150	HLD interview tips: how to structure your answer

Extended Questions (Q143–Q150)

Q143. How do you design a distributed rate limiter?

Difficulty: Hard

Requirements:

- 100M users, 1000 req/sec per user, global (multi-region)
- < 5ms latency overhead, 99.9% availability

Algorithm: Sliding window counter (Redis)

- Key: `rate:{user_id}:{window_start}`
- INCR + EXPIRE per window

Single-region design:

- Request → API Gateway → Rate Limiter (Redis) → Backend
- Redis: sorted set per user (timestamp as score)
- Lua script: atomic check-and-increment

Multi-region (approximate):

- Each region has Redis cluster
- Local decision: allow if local count < local_limit (limit/N regions)
- Sync: background sync every 100ms via cross-region replication
- Accept: slight overage during sync lag (eventual consistency)

Data structures:

- Sliding window: ZSET (score=timestamp, member=request_id)
- Fixed window: STRING with INCR + EXPIRE
- Token bucket: HASH {tokens, last_refill_time}

Handling bursts:

- Token bucket allows burst up to B tokens
- Leaky bucket: smooths output (queue-based)
- Sliding window: most accurate (no boundary burst)

Failure mode:

- Redis unavailable → fail open (allow all) or fail closed (block all)
- Recommendation: fail open with circuit breaker (availability > rate limiting)

Scale numbers:

- Redis Cluster: 1M+ ops/sec
- Latency: <1ms for rate check
- Memory: 100M users × ~100 bytes = 10GB (manageable)

Q144. How do you design a distributed search system?

Difficulty: Hard

Requirements:

- 100TB documents, 10K QPS search, < 200ms latency
- Full-text search, filters, facets, ranking

Architecture:

Ingestion pipeline:

- Data source → Kafka → Indexer → Elasticsearch / OpenSearch
- Indexer: parse, tokenize, normalize, enrich, index

Search architecture:

Client → API Gateway → Search Service → Elasticsearch

Elasticsearch cluster:

Coordinator nodes: route queries, merge results

Data nodes: store shards, execute searches

Master nodes: cluster management (3 nodes, odd number)

Sharding strategy:

Primary shards: $(\text{total_docs} / \text{docs_per_shard}) \approx 1\text{M docs per shard}$

Replicas: 1-2 per primary (HA + read scaling)

Shard routing: $\text{hash}(\text{document_id}) \% \text{num_shards}$

Hot topics: over-shard or use routing key

Query flow:

1. Query parsed → query DSL
2. Coordinator broadcasts to all shards
3. Each shard returns top K locally
4. Coordinator merges, re-ranks, returns top K globally

Relevance ranking:

BM25 (TF-IDF variant): default in Elasticsearch

Learning to rank: ML model on top of Elasticsearch

Vector search: embedding similarity (pgvector or kNN in ES)

Caching:

Query cache: $\text{hash}(\text{query}) \rightarrow \text{results}$ (short TTL, 30s)

Filter cache: cached per filter (boolean queries)

Shard-level request cache: aggregation results

Scale-out:

More data nodes: add shards

More QPS: add replicas (read scaling)

Index rollover: time-based indices (daily/monthly)

Q145. How do you design a real-time collaborative document editor?

Difficulty: Hard

Requirements:

100K concurrent editors, sub-100ms latency for edits

Conflict resolution, offline support, version history

Approach: Operational Transformation (OT) or CRDTs

OT-based (Google Docs approach):

Every edit = operation (insert/delete at position)

Server: serializes operations, transforms concurrent ops

$\text{Transform}(\text{op1}, \text{op2}) \rightarrow \text{op1'}$ applied after op2

CRDT-based (Figma, Notion):

Convergent data structure: any order of operations → same result

No server coordination needed

Types: RGA (Replicated Growable Array), YATA

System components:

Client: local state + pending operations buffer

WebSocket gateway: real-time bidirectional

OT/CRDT server: merge and broadcast operations

Persistence: document state + operation log

Real-time flow:

User types → local apply (optimistic) → send op to server
Server: transform op, persist, broadcast to other clients
Other clients: apply transformed op

Offline support:

Client buffers operations locally
Reconnect: send buffered ops, receive missed ops
CRDT: automatically merges (no conflict)
OT: more complex (transform against all missed ops)

Cursor/presence:

Ephemeral state: Redis pub/sub (not persistent)
Broadcast cursor position via WebSocket
Expire after 30s inactivity

Version history:

Append-only operation log → replay to any point in time
Snapshots every N operations (faster restore)
Git-like branching: copy operation log at branch point

Q146. How do you design a metrics and monitoring system?

Difficulty: Hard

Requirements:

10M time-series, 1M data points/sec, 1-year retention, alerting

Components:

Collection:

Agents: Prometheus exporters, StatsD, OpenTelemetry collector
Pull model (Prometheus): scrape /metrics endpoints every 15s
Push model (StatsD): apps push to aggregator

Storage:

Short-term (3 months): Prometheus TSDB
Long-term (1 year): Thanos or VictoriaMetrics

Thanos: extends Prometheus with object storage (S3)
VictoriaMetrics: single binary, 10x cheaper storage

Data model:

Time-series: {metric_name, labels} → [(timestamp, value)]
Labels: dimensions for filtering (service, env, region, host)
Cardinality: N unique label combinations = N time-series
High cardinality warning: user_id as label = M users × metrics

Query:

PromQL: functional query language
rate(http_requests_total[5m]) → per-second rate
histogram_quantile(0.99, ...) → p99 latency

Alerting:

Prometheus Alertmanager: evaluate rules, deduplicate, route
Alert: PENDING → FIRING → RESOLVED
Routing: team-based, severity-based

Silences: maintenance windows

Visualization:

Grafana: dashboards, panels, variables

Data sources: Prometheus, Loki (logs), Tempo (traces)

Scale numbers:

Prometheus: 1M samples/sec per instance

Thanos Receive: horizontal sharding

VictoriaMetrics: single node handles 10M metrics

Q147. How do you design an e-commerce flash sale system?

Difficulty: Hard

Requirements:

10K concurrent users, 1K items available, order in < 1s

No overselling, fair access

Challenges:

Inventory: 1000 items, 10000 buyers → race condition

Traffic spike: 100x normal traffic in seconds

Fairness: first N users win

Architecture:

Pre-sale phase (before flash sale):

Pre-warm inventory count in Redis: SET flash:inventory 1000

Pre-warm product info in Redis

CDN cache product page (static)

Traffic handling:

Queue / waiting room: excess traffic → queue (SQS + Lambda)

CDN: serve static page with countdown

Rate limit: N requests/user/second

Order flow (with Redis atomic operations):

1. User clicks buy → API checks & deducts inventory

Lua script: if inventory > 0 then DECR(inventory) return 1 else return 0

2. If reserved: create order in DB (async via Kafka)

3. If failed: return "sold out"

Redis Lua (atomic, no race condition):

```
local count = tonumber(redis.call('GET', KEYS[1]))
```

```
if count and count > 0 then
```

```
    redis.call('DECR', KEYS[1])
```

```
    return 1 -- success
```

```
end
```

```
return 0 -- sold out
```

Order confirmation:

Kafka: order events → DB writer → payment → confirmation email

Idempotency: order_id = user_id + product_id + flash_sale_id

Oversell prevention:

Redis atomic decrement (primary guard)

DB: inventory column with CHECK constraint >= 0 (final guard)

Distributed lock for very small inventory (< 10 items)

Scaling:

Redis Cluster: shard by product_id (each shard handles different products)
Multiple API pods: stateless (Redis holds state)
Queue: absorb burst traffic, process at controlled rate

Q148. How do you design a content recommendation system?**Difficulty: Hard****Requirements:**

50M users, 10M items, real-time personalization, < 100ms

Approaches:**1. Collaborative Filtering:**

"Users like you watched X" (user-item matrix)
Matrix factorization (SVD, ALS): $\text{user_vec} \times \text{item_vec} \rightarrow \text{score}$
Challenge: cold start (new users/items)

2. Content-Based Filtering:

"You liked X, here's similar content"
Item embeddings (word2vec, BERT)
Cosine similarity between user history and candidate items

3. Hybrid (production standard):

Combine collaborative + content-based
Multi-armed bandit for exploration/exploitation

System Architecture:**Offline (batch, daily):**

Train model: Spark on S3 data
Generate user/item embeddings
Store in feature store (Redis/DynamoDB)

Near-real-time (hourly):

Update click/watch signals via Kafka
Update user embedding with recent interactions
Candidate generation: ANN search (Faiss, ScaNN)

Online serving (< 100ms):

Candidate generation: retrieve top 500 candidates
- User vector $\times$ item vectors via ANN
- Pre-computed user buckets
Feature retrieval: Redis (user features, item features)
Ranking model: lightweight XGBoost or neural net
Post-processing: filter watched, diversity rules, business rules
Return top 20

A/B testing:

Different models/strategies per user bucket
Metrics: CTR, watch time, satisfaction

Infrastructure:

ANN search: Faiss (Facebook), ScaNN (Google), Redis vector
Feature store: Redis (online), S3 (offline)
Model serving: TensorFlow Serving, TorchServe

Q149. How do you design a global CDN edge network?**Difficulty: Hard****Requirements:**

1B users globally, 10TB/day content, < 10ms latency
99.99% availability, automatic failover

Architecture:**PoP (Points of Presence):**

100+ locations globally, near ISPs
Each PoP: edge servers + local cache + traffic routing

Cache hierarchy:

L1: Edge (PoP) cache → serve most requests
L2: Regional (shield) cache → reduce origin load
L3: Origin → source of truth

Request flow:

User → DNS (anycast) → nearest PoP → hit/miss
Hit: serve from edge cache
Miss → regional cache → miss → origin

Content caching:

Static: images, CSS, JS → long TTL, immutable URLs (hash in name)
Dynamic: HTML → short TTL or ESI (Edge Side Includes)
Private: user data → never cache on CDN

Cache invalidation:

Immediate: API purge (Cloudflare/CloudFront API)
Surrogate keys: tag content, purge by tag
Soft purge: serve stale + async revalidate

Anycast routing:

Same IP announced from all PoPs via BGP
Network routes user to "nearest" PoP (BGP metric)
Fast failover: ~30 seconds (BGP convergence)

Origin protection:

Origin shield: single PoP as shield (origin only receives shield requests)
Rate limiting at edge
WAF: filter attacks at edge (no origin cost)

Metrics:

Cache hit ratio (target: > 90%)
TTFB per region
Error rate per region
Bandwidth served (cost)

Q150. What is HLD production system design checklist?**Difficulty:
Medium****Functional Requirements:**

- All core user journeys documented
- Happy path + error cases covered
- API contract defined (REST/gRPC)
- Data model finalized (entities, relationships)

Non-Functional Requirements:

- Latency targets: p50, p99, p99.9
- Throughput: TPS, concurrent users
- Availability: 99.9% vs 99.99% (cost trade-off)
- Data durability requirements (RPO, RTO)
- Consistency model (strong vs eventual)
- Geographic requirements (single region vs global)

Scalability:

- Stateless services (scale horizontally)
- Database: read replicas, sharding strategy
- Cache: hit ratio targets, invalidation strategy
- Async processing: queues for non-critical paths
- Rate limiting at API gateway

Reliability:

- Single points of failure identified and eliminated
- Circuit breakers for external dependencies
- Retry with exponential backoff
- Graceful degradation (fallback to simpler response)
- Disaster recovery plan (backup, restore, failover)

Security:

- Authentication (JWT, OAuth2)
- Authorization (RBAC, ABAC)
- Data encryption (at rest, in transit)
- Input validation and sanitization
- Audit logging for sensitive operations

Observability:

- Distributed tracing (OpenTelemetry)
- Structured logging with correlation IDs
- Metrics: RED (Rate, Errors, Duration)
- Dashboards and alerts defined
- Runbooks for common failure scenarios